



The Open University

M263 Building Blocks of Software

Unit 7

The class **Set**

Study guide

The four sections of this unit are of similar importance, though Section 2 is a little shorter than the others. There are a few computer activities associated with Sections 3 and 4, but you should not spend more than about 10% of your study time for this unit on practical computing. Section 4 contains some important ideas that are followed up in Units 9 and 10.

This publication forms part of an Open University course. Details of this and other Open University courses can be obtained from the Course Information and Advice Centre, PO Box 724, The Open University, Milton Keynes, MK7 6ZS, United Kingdom: tel. +44 (0)1908 653231, e-mail general-enquiries@open.ac.uk

Alternatively, you may visit the Open University website at <http://www.open.ac.uk> where you can learn more about the wide range of courses and packs offered at all levels by the Open University.

To purchase a selection of Open University course materials, please visit the Open University webshop at www.ouw.co.uk, or contact Open University Worldwide, Michael Young Building, Walton Hall, Milton Keynes MK7 6AA, United Kingdom for a brochure. tel. +44 (0)1908 858785; fax +44 (0)1908 858787; e-mail ouwenq@open.ac.uk

The Open University, Walton Hall, Milton Keynes, MK7 6AA.

First published 2004. Second edition 2008.

Copyright © 2008 The Open University

All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, transmitted or utilised in any form or by any means, electronic, mechanical, photocopying, recording or otherwise, without written permission from the publisher or a licence from the Copyright Licensing Agency Ltd. Details of such licences (for reprographic reproduction) may be obtained from the Copyright Licensing Agency Ltd of 90 Tottenham Court Road, London W1T 4LP.

Edited, designed and typeset by The Open University, using the Open University T_EX System.

Printed and bound in the United Kingdom by The Charlesworth Group, Wakefield.

ISBN 978 07492 2300 7

Contents

Introduction	4
1 Membership and subsets	6
1.1 Set notation	7
1.2 Set membership and propositions	9
1.3 Subsets	11
1.4 Equality of sets	14
2 Operations on sets	17
2.1 Union of sets	17
2.2 Venn diagrams	19
2.3 Intersection of sets	20
2.4 Difference of sets	21
3 The class Set	23
3.1 The power set	24
3.2 Cartesian product of sets	25
3.3 The set <i>Grid</i>	27
3.4 Classes of pairs	29
3.5 The class Set	31
4 Set comprehension	36
4.1 Predicates	36
4.2 Set comprehension	38
4.3 Using set comprehension	42
Summary of Unit 7	45
Solutions to the exercises	46
Index	48

Introduction

The idea of a set provides a particularly simple way to describe a collection of elements. Despite this simplicity, sets are fundamental to mathematics. Sets, and the operations on them that we shall describe in this unit, also form a powerful tool in the theory of computing.

In Unit 1, we mentioned various forms of collection: in particular, sets and sequences. Recall that in a sequence the order in which the elements of the collection appear is significant, as is the repetition of an element. In a set, we are not concerned with the order in which the elements are given, nor are we concerned with any repetitions of an element. The key question that we may ask about a set is this: “is a given item in the set?”

In this unit, we shall introduce various processes on sets. We will then incorporate these processes in a class **Set of X**, thus making them available in code.

In Unit 6, we discussed collection classes based on sequences, with the aim of helping you to become familiar with the way these classes can be used in code. We can discuss sets in a similar way, and indeed our main emphasis here will be on describing sets and operations on sets in a way that corresponds with the implementation of the class **Set of X**. However, while most programming languages provide types corresponding to **Vector of X**, many do not provide a type corresponding to **Set of X**. Our provision of the class **Set of X** is mainly to help you to consolidate your ideas about sets.

The notion of set plays a wider role in mathematics and computing science than simply as a particular form of collection underlying a class. Sets are a key part of the language used when discussing the fundamentals of computing. Much of the importance of the idea of a set lies in that point, although such use of sets will not feature in this unit.

As with other forms of collection, we shall not mix items of different types in a variable of type **Set of X**. The **X** in **Set of X** shows the type of item contained in a set object. So, for example, we may form a set containing integers, or a set containing characters, but not a set containing a mixture of integers and characters.

In this unit, we shall use the word “set” in a way that can usually be interchanged freely with “a variable of the type **Set of X**”. However, in mathematics in general, the term “set” is used with no direct relation to a variable in code. If you read other texts, they are likely to take a view of sets corresponding to the use of the term in general mathematics. The concepts are not different, but we shall discuss sets in a way that emphasises features that are of particular relevance to the view of a set as a collection that can be used in code. For example, in mathematics in general, one would have no problem with discussing infinite sets. Recall that as a mathematical entity the set of all integers, *Int*, is infinite. But, as noted in Unit 3, any implementation of a type **Int** in code will need to have a finite set of integers as its data set. For similar reasons, any variable of type **Set** will contain a finite number of members. In this unit, we shall only consider finite sets.

We shall not keep repeating this comment!

There are two main ways of describing a set, both of which were mentioned briefly in Unit 1. The first is simply to write down all the elements of the set. This is known as **set enumeration**. For example, we may write the set containing the integers between 1 and 9 (inclusive) as:

$$\{1, 2, 3, 4, 5, 6, 7, 8, 9\}.$$

The second way is to give a property or properties that elements of the set must have. We may write this same set as:

$$\{x \text{ in } Int : 0 < x < 10\}.$$

This second method of describing a set is known as **set comprehension**.

We introduced this notation in Unit 1, but there we wrote $x \in Int$ rather than $x \text{ in } Int$. However, the symbol $\in$ may be used in two related but different ways, and we want to distinguish between them. In set comprehension, the first step is to state the type of object contained in the set. This process of typing plays a similar role to a code statement of the form **var x in Int** , which gives the type of a variable; hence our use of **in** rather than $\in$ for set comprehension.

We use **in** when giving the type of a variable (here x). We use $\in$ to show set membership (see p. 9). A fuller discussion is given on p. 38.

In set comprehension, the type is given explicitly, but whichever way we describe a set, all the items in it will be of the same type. Either way, we give the type of the items by reference to a set that we already know, which we will call a **predefined set**. A predefined set will usually be the data set of some type, such as *Int*, as in the example above, or *Char*.

In mathematics in general, sets are more often described using comprehension than enumeration. However, our class **Set** is based on enumerated sets, so we shall give prominence to sets given in that way.

In Section 1 we look at some basic ideas relating to sets. In particular, we consider recognition of whether or not a given item is in a set, and of whether or not two sets are identical. In Section 2 we consider operations on sets that produce new sets from given ones. Our examples in these sections will mostly relate to sets of integers, but the concepts are relevant to sets with members of any type.

In Section 3 we specify the class **Set of X** and use its methods to illustrate the concepts of the first two sections. In Section 4 we consider set comprehension; that is, sets described using conditions rather than by enumeration.

If you look for “Set Theory” at the “History of Maths” web site at St Andrews University you will find the following:

“The history of set theory is rather different from the history of most other areas of mathematics. For most areas a long process can usually be traced in which ideas evolve until an ultimate flash of inspiration, often by a number of mathematicians almost simultaneously, produces a discovery of major importance. Set theory however is rather different. It is the creation of one person, Georg Cantor. ... In 1874 Cantor published an article in Crelle’s Journal which marks the birth of set theory.”

See <http://www-history.mcs.st-andrews.ac.uk/history/>

Cantor’s concept of associating a single entity, a set, with a collection of elements, was a startlingly new idea and took some time to be accepted. But by 1902 set theory was beginning to have a major impact in mathematics.

1 Membership and subsets

Consider the following list of integers:

2, 1, 2, 2, 5, 1, 4, 5, 5

We are going to describe an entity that can be derived from this list. The information that we require the entity to store is whether or not an integer is in the list. For a small collection such as this there is a simple notation for such an entity, namely $\{2, 1, 5, 4\}$.

The curly brackets mean that we regard $\{2, 1, 5, 4\}$ as a single entity, called a **set**. This set is said to be derived by enumeration from the original list. Since we are not concerned with the order in which the items appear, $\{1, 2, 4, 5\}$ is an equally good representation of this set.

A set has the following essential features.

- There is no need to repeat elements.
- A set does not depend on the order in which the elements are written between the curly brackets.

Given any list of items, we can derive the corresponding set by deleting any repeated items from it, and writing the remaining items between curly brackets (where the order in which they are written is of no significance).

A set is given **by enumeration** if we write out each of the members of the set. For example, the set $\{2, 1, 5, 4\}$ is given by enumeration, as is the set $\{‘q’, ‘w’, ‘e’, ‘r’, ‘t’, ‘y’\}$. We noted in the introduction that we will not mix items of different types within a set. This is not an important issue at this point, where the sets we consider are enumerated. But it is important to the way we will develop sets in this unit, and in particular to the way we will form a class based on sets. The sets we consider in this section will either consist entirely of integers (from *Int*), or will consist entirely of characters (from *Char*).

Activity 1.1

Which of the following represent the set derived from the list of integers 2, 4, 3, 2, 1, 1, 1?

- (a) $\{2, 4, 3, 1\}$. (b) $\{1, 2, 3, 4\}$. (c) $\{2, 3, 1\}$. (d) $\{1, 2, 3, 4, 5\}$. (e) $\{1, 1, 2, 3, 4\}$.

Discussion Each of (a), (b) and (e) represents the set derived from the list. (a) This representation has been formed in what is probably the simplest way of deriving the set from the list. This is to start at the leftmost integer in the list and move to the right, writing down one copy of each new integer that you come to. (b) This is also a set representation of the list. It does not matter that we have put the numbers in a different order. (c) This does not show that 4 is in the list and so it is not a correct representation. (d) This set contains the integer 5, which is not in the list, so this set is not a correct representation of the list. Both (c) and (d) are sets, but neither of them is a set representation of the given list. (e) This is a set representation of the list but we would not in practice include the repeated element.

1.1 Set notation

A set can be given an identifier (that is, a name). We will adopt the convention that a set identifier will start with a capital letter, and may sometimes be just a single letter, such as A , B or C .

For example, by writing $A = \{2, 1, 3, 5, 7\}$ we give this set the identifier A . Here, we are thinking of the set as a fixed entity. That is, we are not using A as the identifier of a variable whose state is a set, which might change. We are associating the identifier A with a specific and constant set value. When we use such an identifier, this use is usually “local” to a particular example or activity. The same identifier may be used later for a different set.

However, there are certain notations that we will invariably use to identify certain standard sets. Examples of these are *Char* and *Bool*, which are the data sets of the data types **Char** and **Bool**.

Both of these are finite sets, and can be enumerated (all the elements written down). To do this with a small set such as *Bool* is no problem. We have $Bool = \{\mathbf{true}, \mathbf{false}\}$. The set *Char* is much larger, but it was given in its entirety (all 95 elements) in Unit 1 and elsewhere.

In Unit 1 you were also introduced to *Int*, the set of all integers. This set is not finite. However when the integers are implemented on a computer as the data type **Int**, the data set underlying this will be finite. The actual collection of integers in this set depends on the language we are working with. For instance, in JavaScript, the largest integer in the set has the value somewhere around 10^{15} . Suppose that *MAXINT* is the largest integer implemented and *MININT* is the smallest integer implemented, and we call the set of implemented integers *ImpInt*. Then we can write this set as

$$ImpInt = \{n \text{ in } Int : MININT \leq n \leq MAXINT\}$$

where *Int* is the infinite set of all integers.

This is a very large set, and is not one that we would attempt to enumerate on paper, or even ask a machine to enumerate electronically! You may sometimes see a notation such as that below used to describe such a set:

$$MININT, \dots, -2, -1, 0, 1, 2, 3, \dots, MAXINT.$$

This appears to list the elements of the set, but only does so by implication: the members of the set have certainly not all been written down. The three dots, known as an ellipsis, are a way of indicating that, written this way, all the other elements of the set can be deduced in an orderly process by following the pattern of those that have been written down.

There are two sets that we shall use frequently in examples, so we shall not change the definitions of these sets, which are given below. The first set has identifier *SmInt* (short for “small integer set”) and is enumerated as follows:

$$SmInt = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}.$$

Many of the examples that follow in this section are based on the elements of *SmInt*. This small set is adequate to illustrate key concepts and notation.

The second example set has identifier *LInt* (short for “large integer set”) and is the set of integers between -100 and 100 inclusive, that is:

$$LInt = \{n \text{ in } Int : -100 \leq n \leq 100\}.$$

We could enumerate this set on paper, but doing so would not be particularly informative. To decide whether a given integer n is in *LInt*, it is much easier

Programming languages often have more than one type with a data set containing integers. The difference is usually in the size of integers allowed. For example, Java has classes **byte**, **short**, **int** and **long** covering four different integer data sets. The size of integer allowed is determined by how many bytes are required to store the integer. Using a class with a small data set can save on both space and time.

to determine whether n satisfies the condition $-100 \leq n \leq 100$ than it is to check through a list of over two hundred integers to see whether n appears in that list.

Given an empty list of elements (a list that contains nothing) the set enumeration of such a list is written simply as $\{\}$. This set, with no elements in it, is referred to as the **empty set**.

The **cardinality** of a set is the number of elements in the set. For a finite set A , we will denote its cardinality by $CARD(A)$. (So $CARD$ is a function with output set Int .)

For example, $CARD(\{2, 1, 3, 5, 7, 4, 8\}) = 7$.

You met this notation in Unit 1, where we noted that some texts use the symbol $\emptyset$ for the empty set.

Activity 1.2

Determine the following.

(a) $CARD(Bool)$. (b) $CARD(\{\})$. (c) $CARD(SmInt)$. (d) $CARD(LInt)$.

Discussion For (a) and (c) we just count the elements to get 2 and 10 respectively. For (b), we have $CARD(\{\}) = 0$, since the empty set has no elements. For (d), the cardinality of $LInt$ is 201. In $LInt$, there are 100 negative numbers, 100 positive numbers, and the integer 0.

Activity 1.3

Let Y be the set of characters appearing in the string “aardvark”.

(a) Give an enumeration of Y . (b) What is $CARD(Y)$?

Discussion

(a) The set is enumerated below. You may have the characters in a different order, but avoid repeating characters.

$\{\text{'a'}, \text{'r'}, \text{'d'}, \text{'v'}, \text{'k'}\}$

(b) The enumeration given in (a) shows that Y contains five characters, so $CARD(Y) = 5$.

We avoid repeating characters in a set enumeration because this is potentially confusing.

1.2 Set membership and propositions

In general mathematics, definitions of sets may be given in a variety of ways, and in some cases it can be very difficult to determine which elements lie in the set. However, with the forms of set definition we consider in this unit, it is not a difficult task to determine whether an element lies in a set. In particular, this is a straightforward task for a small set given by enumeration. In that case, the task of determining whether an element is in the set is one of inspecting the elements in some systematic way.

We will shortly set out a procedure for addressing this task in a way that might be implemented on a computer. First, we introduce some terminology that will be used extensively in this unit (and in Unit 9).

Consider the assertions s and t shown below.

s : 3 is an element of the set $\{2, 1, 3\}$.

t : 4 is an element of the set $\{2, 1, 3\}$.

The first of these looks to be **true**: 3 is an integer and it is in the set $\{2, 1, 3\}$. The second looks to be **false**. Although 4 is an integer it is not shown in the set. Each of these assertions is an example of a **proposition**.

Propositions will be discussed in greater detail in Unit 9.

A proposition has a **truth value**, by which we mean we can determine unequivocally either that the assertion is **true** or that it is **false**. In the examples above, the proposition s has the truth value **true** (3 is, by inspection, an element of the set $\{2, 1, 3\}$), while the proposition t has the truth value **false** (by inspection, 4 is not listed in the set enumeration $\{2, 1, 3\}$).

In passing, note that you have met other examples of propositions in earlier units. For instance $2 < 5$ and $2 \geq 5$ are propositions. These have the truth values **true** and **false** respectively. These are Boolean values returned by the comparisons $<$ and $\geq$. We shall discuss the relationship between propositions and such conditions later.

In Unit 1, we introduced the symbol $\in$, which replaces the term “is an element of the set”, as in $2 \in \text{SmInt}$. Using this notation the two propositions above are written as follows.

s : $3 \in \{2, 1, 3\}$.

t : $4 \in \{2, 1, 3\}$.

The first has the truth value **true**, the second has the value **false**.

For $x \in A$ to be a correctly formed proposition, we will require it to be type consistent. For example, if A is a set containing integers, then the expression $x \in A$ is a proposition only if x is an integer. For example, we do not regard ‘ a ’ $\in \text{SmInt}$ as a proposition, since ‘ a ’ is a character and SmInt is a set of integers.

For a set A described by enumeration, and an item x of the appropriate type, the procedure for determining the truth value of a proposition of the form $x \in A$ is simple. One just needs to inspect all the elements of the set A , and compare them to x .

Activity 1.4

- (a) Suppose that A is the set $\{2, 1, 5, 4\}$. Write down the truth value of each of the following propositions.
 (i) $6 \in A$. (ii) $2 \in A$. (iii) $1 + 1 \in A$.
- (b) Write down the truth value of each of the following propositions.
 (i) $1 \in \{\}$. (ii) $0 \in \{\}$.
- (c) Let Y be the set of characters appearing in the string “aardvark”. Write down the truth value of each of the following propositions.
 (i) ‘k’ $\in Y$. (ii) ‘A’ $\in Y$. (iii) “dv” $\in Y$.

Discussion

- (a) The proposition in (i) has the truth value **false** as 6 does not appear in the set enumeration. The propositions in (ii) and (iii) both have value **true**. That (iii) is **true** may not have been immediately obvious. However, one of the elements of the set A is the integer represented by the symbol 2. Both 2 and $1 + 1$ represent this integer. It is not the equivalence of the symbols we are checking, but the equivalence of the integers.
- (b) Since the empty set contains no elements, it cannot contain the integer 1, nor for that matter any other element, even 0. So the truth value of both these propositions is **false**.
- (c) (i) This proposition is **true**. This set Y was enumerated in the discussion of Activity 1.3, and the character ‘k’ does appear in the set. (ii) Remember that ‘A’ and ‘a’ are different characters. ‘A’ does not appear in the set Y , and this proposition is **false**. (iii) This is not a proposition, since Y is a set containing characters, and “dv” is not a character (it is a string). If we regard $\in$ as a function returning a value in **Bool**, then we need the input values to be of the correct type for the signature. The expression in the question is not type consistent, so we do not regard it as a proposition, and we do not assign any truth value to it.

We will often shorten “has the truth value” to “is”.

Determining the truth value for a set membership proposition by hand is a straightforward task for a small set given by enumeration. We now set out a strategy for doing this. This strategy may seem obvious, but it characterises a procedure that can be implemented by a computer.

Strategy 1.1

To determine the truth value of $x \in A$.

- Compare x with any element of A . If they are the same, then the value of the proposition is **true**.
- If they are not the same, cross out the element of A that you have examined.
- Repeat the process until every element of A has been examined.
- If x is not found to be the same as any of the elements of A , then the value of the proposition is **false**.

All the strategies that we set out in this section and the next are for sets given by enumeration. They also require type consistency. We shall not repeat this each time.

As a simple example, consider the strategy applied to determining the value of the proposition $6 \in \{2, 1, 5, 4\}$ (see Activity 1.4 (a)(i)).

- Choose any element in $\{2, 1, 5, 4\}$, say 5. Compare 6 to 5.
This is not the same, so cross out the 5 to get $\{2, 1, \cancel{5}, 4\}$
- Choose an element that remains, say 2. Compare 6 to 2.
This is not the same, so cross out the 2 to get $\{\cancel{2}, 1, \cancel{5}, 4\}$
- Choose an element that remains, say 1. Compare 6 to 1.
This is not the same, so cross out the 1 to get $\{\cancel{2}, \cancel{1}, \cancel{5}, 4\}$
- Choose 4, the only remaining element. Compare 6 to 4.
This is not the same, so cross out the 4 to get $\{\cancel{2}, \cancel{1}, \cancel{5}, \cancel{4}\}$

Hence $6 \in \{2, 1, 5, 4\}$ is **false**.

Of course a visual check is almost instantaneous for such a small set. When checking a proposition about set membership yourself, there is no need to try to follow the procedure set out in Strategy 1.1. Do it in any way you find convenient. The above procedure outlines how the truth value of a proposition about set membership can be determined in a way that could be expressed in code, and that would work for any enumerated set, no matter how large.

Note also that the comparison is not always a straightforward visual check; for instance, $DIV(526, 260)$ is a member of the set $\{2, 1, 5, 4\}$.

You do not have to work systematically from left to right, even if it seems the most logical thing to do, since the order in which the elements are written between the brackets is of no importance.

A similar comment applies to all the strategies that we will give in this unit.

1.3 Subsets

All the elements of the set $\{2, 1, 5, 4\}$ are also to be found in the set $SmInt = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$. We say that the set $\{2, 1, 5, 4\}$ is a **subset** of $SmInt$.

Consider the following assertions.

- (1) $\{2, 1, 5, 4\}$ is a subset of $SmInt$
- (2) $\{2, 1, 5, 4, 33\}$ is a subset of $SmInt$

These assertions are propositions. (1) is **true**, while (2) is **false** (since the integer 33 is not an element of $SmInt$).

We replace “is a subset of” by the symbol $\subseteq$, so that these propositions can also be written as below.

- (1) $\{2, 1, 5, 4\} \subseteq SmInt$
- (2) $\{2, 1, 5, 4, 33\} \subseteq SmInt$

Note that, in many texts, statements such as $7 < 10$ or $\{2, 1, 5, 4\} \subseteq SmInt$ would be written with the implication that they are **true**. However, as noted in Unit 1, operations such as $<$ and $\subseteq$ can be associated with functions that return a value in **Bool**. This insight is necessary if we are to make such operations available in code, and so it is a view that we emphasise in this course. So we regard any expression such as those above as a proposition; that is, as something that may be either **true** or **false**. (To assign the appropriate truth value to a proposition, we use our knowledge of the context in which the proposition is formed.)

In general, we say that a set B is a **subset** of the set A if every member of B is also a member of A . Since $\subseteq$ is a relationship between sets, we regard the expression

$$B \subseteq A$$

as a valid proposition only if each of A and B is a set, where these sets contain members of the same type.

Activity 1.5

Write down, where possible, the truth value of each of the following.

- (a) $\{2\} \subseteq \{0, 1, 2\}$
- (b) $\{2, 1, 0\} \subseteq \{0, 1, 2\}$
- (c) $SmInt \subseteq LInt$
- (d) $LInt \subseteq SmInt$
- (e) $2 \subseteq \{0, 1, 2\}$

Discussion The proposition in (a) is **true**. The only member of the set $\{2\}$ is the integer 2, and this is in the set $\{0, 1, 2\}$. The proposition in (b) is also **true**. Each element of the left-hand set is to be found in the right-hand set. Note here that the notion of “subset” includes identical sets; that is, a set is a subset of itself. For (c) and (d) we need to recall what the elements of the two sets are. The elements of $SmInt$ are the integers from 0 to 9, and all of these are certainly in $LInt$. So the proposition in (c) is **true**. For (d), we only need to note that the integer 33 (for example) is in $LInt$ but is not in $SmInt$. (There are many suitable examples.) Hence (d) is **false**. In (e), 2 is an integer, not a set, and so writing $2 \subseteq \{0, 1, 2\}$ is not type consistent. Consequently, the expression in the question is not a proposition. (Either of the expressions $2 \in \{0, 1, 2\}$ or $\{2\} \subseteq \{0, 1, 2\}$ would give a correctly formed proposition, and each is **true**.)

The terminology “sub” is not quite as one might expect.

A mechanical procedure for determining whether one set is a subset of another can be described as below.

Strategy 1.2

To determine the truth value of $B \subseteq A$.

1. Consider any element of the set B , x say, (so that $x \in B$ is **true**).
2. Apply Strategy 1.1 to determine the truth value of $x \in A$.
3. If $x \in A$ is **false** (so that x is not a member of A) then $B \subseteq A$ is **false** and the process stops.
4. If $x \in A$ is **true**, then repeat the process with a different element of B .

If all the elements of B have been considered and the process has not stopped at Step 3, then $B \subseteq A$ is **true**.

This strategy is only applicable to sets given by enumeration. The following result applies to any sets, however they are described.

Result 1.1

Suppose that, for every x of the correct type: if $x \in B$ is **true** then $x \in A$ is **true**. We can deduce that $B \subseteq A$ is **true**.

Conversely, if we can find an x of the correct type such that $x \in B$ is **true** and $x \in A$ is **false** then $B \subseteq A$ is **false**.

The underlying logic of these statements is quite complicated. We will look at the logical ideas involved in Unit 10.

For example, to show that $\{4, 5\}$ is a subset of $SmInt$, we need only to consider the two propositions $5 \in SmInt$ and $4 \in SmInt$. Since both are **true** it follows that the proposition $\{4, 5\} \subseteq SmInt$ is **true**. Conversely, as in Activity 1.5(d), we need only to produce one example of an integer that is in $LInt$ and is not in $SmInt$ to show that $LInt \subseteq SmInt$ is **false**.

EXAMPLE 1.1

Write down the truth value of the proposition $\{\} \subseteq \{2, 1, 3\}$.

There are no elements in the empty set. So we can certainly assert that every element in $\{\}$ is also in $\{2, 1, 3\}$. This proposition is **true**.

A similar argument shows that the empty set is a subset of every set.

This result indicates that set theory is a little more complicated than might appear at first sight.

Both Strategies 1.1 and 1.2 are for sets given by enumeration, that is, sets with a finite number of elements that are given in an explicit list. For such sets it is possible to consider the entries one by one. However, it is possible to define sets in ways other than by enumeration. For example, the mathematical set of all integers, Int , certainly cannot be given by enumeration as it is not finite. In such cases, we cannot use the strategies given above to test for the relations of set membership or set containment, since we cannot work through a set by considering each element in turn.

For instance, let $TenPlus$ be the set of all integers greater than 10, and $PosInt$ be the set of all integers greater than 0. These are both infinite sets. However, it would seem reasonable to assert that $TenPlus$ is a subset of $PosInt$. We cannot follow Strategy 1.2 in this case, because the strategy will never be completed. (There will always be another element to consider!) In such a case, we can use Result 1.1. Any integer greater than 10 is certainly greater than 0. So we know that if $x \in TenPlus$ is **true** then $x \in PosInt$ is **true**. (The full chain of argument is: if $x \in TenPlus$ is **true**, then $x > 10$ is **true**, so then $x > 0$ is **true**, so then $x \in PosInt$ is **true**.)

We are rarely interested in infinite sets from the computing point of view so the sets we shall deal with will all be finite. However the sets can still be very large. They are often large enough to make enumeration inconvenient, particularly if working “on paper”. For instance, we would not want to write down an enumeration of the finite set $LInt$; it would be simply too tedious. However, we do know what constitutes an element of $LInt$ and we can say whether a given element is in it. When dealing with such sets we will need the more general form of the subset relationship as given in the Result 1.1, since that does not require us to work “element by element”.

1.4 Equality of sets

We now turn our attention to equality of sets. For sets given by enumeration, we need to take care to distinguish between the set and the way in which it is enumerated. For example, the sets $\{2,1,0\}$ and $\{0,1,2\}$ are equal. They are identical sets, since they have exactly the same elements (remember that the order in which we write the elements between the brackets is of no consequence).

We can give the following definition of the idea of equality of sets.

Two sets A and B are **equal** if they contain exactly the same elements.

We will write $A == B$ for the proposition asserting that the sets A and B are equal. As with other propositions (such as $A \subseteq B$), $A == B$ is something that can take the value **true** or **false**.

Activity 1.5(b) concerned the proposition $\{2, 1, 0\} \subseteq \{0, 1, 2\}$. This is **true** since all the elements of $\{2, 1, 0\}$ are to be found in $\{0, 1, 2\}$.

Now $\{0, 1, 2\} \subseteq \{2, 1, 0\}$ is also **true**. So we see that each of the sets $\{0, 1, 2\}$ and $\{2, 1, 0\}$ is a subset of the other. To say that each set is a subset of the other is equivalent to saying that the two sets have exactly the same elements and hence are equal.

We can give a strategy for showing that two sets are equal as follows.

Strategy 1.3

To determine whether $A == B$.

- Use Strategy 1.2 to determine whether $A \subseteq B$ is **true**.
- If $A \subseteq B$ is **true** then determine whether $B \subseteq A$ is **true**.
- If $B \subseteq A$ is also **true** then $A == B$.

Strategy 1.3 draws on Strategy 1.2, and so, like that, is a strategy applicable to sets given by enumeration. Even if sets are not given by enumeration, we can use the same approach to test for equality. Result 1.2 expresses this idea formally. To use it, we would need to use Result 1.1 to determine whether the subset relation holds.

Result 1.2

$A == B$ has the same truth value as $(A \subseteq B) \wedge (B \subseteq A)$.

Remember that $\wedge$ is read as “and”. The proposition $(A \subseteq B) \wedge (B \subseteq A)$ is **true** if (and only if) both the propositions $A \subseteq B$ and $B \subseteq A$ are **true**. Note that when showing that two sets A and B are equal we need to carry out two steps:

show that $A \subseteq B$ is **true**;
show that $B \subseteq A$ is **true**.

We use a single equals sign, $=$, when assigning an identifier to a particular set, as in, for example, $SmInt = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$.

$(A \subseteq B) \wedge (B \subseteq A)$ is called a composite proposition. We shall discuss composite propositions in Unit 9.

Activity 1.6

For which of the following sets does the proposition $A == B$ have value **true**?

- (a) $A = \{1, 2, 3\}, B = \{2, 1, DIV(9, 3)\}$.
- (b) $A = \{1, 2\}, B = \{1, 2, 3\}$.
- (c) $A = \{\}, B = \{2\}$.
- (d) $A = \{'1', '2'\}, B = "112"$.

Discussion Only the proposition in (a) is **true**.

- (a) $DIV(9, 3)$ is another way of denoting the integer 3. So both these sets are subsets of each other, and they are equal.
- (b) $\{1, 2\} \subseteq \{1, 2, 3\}$ is **true** but $\{1, 2, 3\} \subseteq \{1, 2\}$ is **false** as 3 is not an element of $\{1, 2\}$. So these sets are not equal.
- (c) The empty set is a subset of $\{2\}$ (it is a subset of any set), but $\{2\}$ is not a subset of the empty set. ($\{\}$ has no elements, and so certainly cannot have 2 as an element.) Hence these sets are not equal.
- (d) Here we are not dealing with two sets. "112" is not a set, it is a string. (We could form a set containing the characters appearing in this string, but that would be a different entity.) Hence $A == B$ is not a well formed proposition here, and so is neither **true** nor **false**.

We conclude this section by mentioning one final concept. Suppose that $A \subseteq B$ is **true**, but $A == B$ is **false**. We then say that A is a **proper subset** of B . For instance $\{2, 1\}$ is a proper subset of $\{1, 2, 3\}$, since all the members of $\{2, 1\}$ are in $\{1, 2, 3\}$, but $\{1, 2, 3\}$ contains a member (3) that is not in $\{2, 1\}$. We use the symbol $\subset$ rather than $\subseteq$ to show this – note that the line underneath is missing. This corresponds with the “less than” ($<$) and “less than or equal” ($\leq$) operation symbols for integers. Thus we can write, for example, $\{2, 1\} \subset \{1, 2, 3\}$.

The proposition $A \subset B$ is harder to work with than is $A \subseteq B$. Not only do we have to show all the elements in A are in B , but we also have to show that there is an element in B that is not in A .

Aside

Equality is so simple a notion that it is easy to overlook its importance. For example, all the strategies we have described in this section require that we can compare two items to see whether they are the same. To implement processes such as set membership and the subset relationship, we will need to have available a test of equality for the type of item contained in the sets. Our examples have covered only integers and characters. Both the data types **Char** and **Int** have a type function of equality (with identifier $==$), so testing set members for equality is no problem in those cases.

We have not given methods for testing equality in every class that we have specified. (For example, the specification of **Spot** does not include a test of equality.) It would be possible to supply a test of equality for objects of any class. The natural approach when comparing two objects of the same class for equality is to return **true** if they have the same value for every attribute, and **false** otherwise. Since the attributes may have the type of

some previously defined class, we can only be sure that this is possible if we supply every class with an equality test.

Provision of equality tests raises a subtle problem in the context of class extension. For example, consider the class **ColourSpot**. We would want to treat two objects of type **ColourSpot** as equal if they have the same value for all three attributes: *xPos*, *yPos* and *colour*. But **ColourSpot** extends **Spot**, and so **ColourSpot** would also inherit an equality test from **Spot**. That would compare only two of the attributes: *xPos* and *yPos*, and so is a different process! This conflict can be resolved, but it raises issues that we do not want to discuss in this course.

End of Aside

Objectives for Section 1

After studying this section you should be able to do the following.

- ☐ Given a list of integers or characters, write down the corresponding set given by enumeration.
- ☐ Recognise and use the terminology: proposition, element of a set, empty set, cardinality, subset, proper subset, equal sets.
- ☐ In specific cases give the truth value of propositions of the following forms: $x \in A$; $A \subseteq B$; $A == B$; $A \subset B$.

Exercises on Section 1

Exercise 1.1

Let Q be the set of characters in the string “didgeredoo”.

- (a) Give the set Q by enumeration.
- (b) Where possible, give the truth value of each of the following.
 - (i) ‘o’ $\in Q$. (ii) ‘Q’ $\in Q$. (iii) “o” $\in Q$.

Exercise 1.2

What is the truth value of each of the following propositions?

- (a) $\{1, 2\} \subseteq \{1, 2, 3\}$
- (b) $\{1, 2\} \subset \{1, 2, 3\}$
- (c) $SmInt \subset SmInt$
- (d) $\{\} \subseteq \{\}$
- (e) $\{\} \subset \{\}$

Exercise 1.3

Let Y be the set of characters appearing in the string “adder”, and X be the set of characters appearing in the string “dread”. What is the truth value of each of the following propositions?

- (a) $Y \subseteq X$. (b) $Y == X$. (c) $Y \subset X$.

Exercise 1.4

List all the proper subsets of $\{4, 6, 1\}$.

2 Operations on sets

In this section we consider some important processes on sets. We shall consider three different ways of combining two sets A and B , which contain elements of the same type.

- The first forms the set consisting of all the members of A and all the members of B (and is called the **union** of the two sets).
- The second forms the set consisting of all the members that A and B have in common (and is called the **intersection** of the two sets).
- The third forms the set consisting of the members of A that do not appear in B (and is called the **difference** of the two sets).

Each of these processes combines two sets to form a new set. So each is a binary operation on sets. We noted in Unit 1 that we can use infix notation for a binary operation, and we shall introduce an infix notation for each of these binary operations on sets.

2.1 Union of sets

The first operation on sets that we shall consider is **set union**. This combines two sets to form a new set that consists of all the members of the two sets. The infix notation used for the union operation is the symbol $\cup$.

For example, the union of the sets $\{1, 2, 3\}$ and $\{3, 4, 5\}$ is formed by writing down all the elements of both sets and removing any repetitions. So the union of these two sets is the set $\{1, 2, 3, 4, 5\}$. We write

$$\{1, 2, 3\} \cup \{3, 4, 5\} = \{1, 2, 3, 4, 5\}.$$

We have noted already that we do not mix items of different types within the same set. This means that when we combine two sets using any of the binary operations we shall introduce, the two sets being combined must contain members of the same type. So we might form the union of two sets containing integers, or the union of two sets containing characters, but we would not use the union operation to combine a set containing integers with a set containing characters.

Where we have two sets given by enumeration, an enumeration of the union can be found using the strategy given below.

Strategy 2.1

To find the union $A \cup B$.

- Write down together all the elements of the set A and all the elements of the set B .
- Remove any repetitions.

Activity 2.1

For each union below, state whether it can be formed. Where it can be formed, enumerate the set and write down its cardinality.

- (a) $\{2, 1\} \cup \{4, 3\}$. (b) $\{1, 2\} \cup \{3, 2, 1\}$. (c) $\{'q', 'a', 'z'\} \cup \{'q', 'u', 'i', 'z'\}$.
 (d) $\{1, 2\} \cup \{'a', 'b'\}$.

Discussion In each of (a), (b) and (c), the union combines two sets whose members are of the same type, so each of these three unions can be formed. We can form these unions using Strategy 2.1.

- (a) The members of the two sets are: 2, 1, 4, 3. Hence the union is the set $\{2, 1, 4, 3\}$. This has cardinality 4.
 (b) The members of the two sets are: 1, 2, 3, 2, 1. So, removing repetitions, the union is $\{1, 2, 3\}$. This has cardinality 3.
 (c) The members of the two sets are: 'q', 'a', 'z', 'q', 'u', 'i', 'z'. So the union is $\{'q', 'a', 'z', 'u', 'i'\}$. This has cardinality 5.
 (d) In this case we have two sets containing elements of different types: the first is of elements from *Int*, and the second has elements from *Char*. We will only form unions of sets that contain the same type of elements. Hence (d) is not defined.

With a set given by enumeration, the predefined set from which it is drawn is not given explicitly. In examples, assume that sets are drawn from a predefined set that is either *Int* or *Char* (unless otherwise stated).

In Activity 2.1(b), notice that the first set is a subset of the second. In such a case, the resulting union is just the second set, since the first set adds no new elements to the union.

Another point to notice about the examples in Activity 2.1 concerns the cardinality of the union. This is at least as big as the cardinality of each of the two sets being combined. However, the cardinality of the union is never larger than the sum of the cardinalities of the two sets being combined. For example, in (c), $3 + 4 \geq 5$.

Activity 2.2

If A is a set, explain why $A \cup \{\} == A$ will always be **true**.

Discussion There are no members in the empty set $\{\}$ to add anything to the union, so $A \cup \{\}$ will be the same as the set A , for any set A .

Strategy 2.1 shows how we can form the union of two sets given by enumeration. How might we find the union for sets not described in that way? An item x of the appropriate type will be in the union of sets A and B if x is a member either of A or of B . This is in effect the definition of the idea of union for sets in general. It corresponds to the following result, which concerns propositions about membership of a union. Recall that the symbol $\vee$ is read as “or”, and this infix operation combines two Boolean values and returns **true** if either or both of them is **true**.

Result 2.1

$x \in (A \cup B)$ has the same truth value as $(x \in A) \vee (x \in B)$.

2.2 Venn diagrams

Venn diagrams give a simple graphical representation of sets, and can be used as an aid to visualising some of the operations on sets. We introduce them here for that purpose only. They are not important in themselves and cannot be used to prove any result concerning sets.

Figure 2.1 gives an example of a Venn diagram. It shows two sets taken from $SmInt$. (The outer rectangle in the Venn diagram shows the predefined set from which the elements of each set are drawn. For convenience, we will draw diagrams with predefined set $SmInt$.) The elements of $SmInt$ are shown in a closed rectangle (distributed in a way that has no significance). Subsets of $SmInt$ are depicted by enclosing their elements. In Figure 2.1, the sets $A = \{2, 1, 3\}$ and $B = \{4, 3, 5\}$ are shown.

Figure 2.1 shows that the element 3 belongs to both these sets, but that neither set is a subset of the other. Notice how the two sets divide $SmInt$ into four distinct areas. These are: elements that are in A but not in B (1, 2), elements that are in B but not in A (4, 5), elements that are in both A and B (3), and elements that are in neither A nor in B (0, 6, 7, 8, 9).

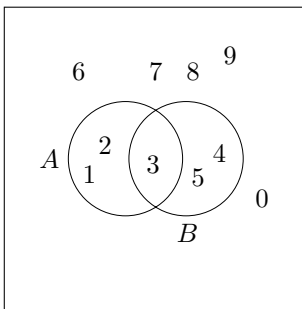


Figure 2.1

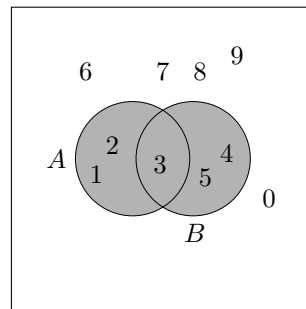


Figure 2.2 $A \cup B$ is shaded.

Particular sets may be shown by shading. For example, the Venn diagram for the union of these same sets A and B is shown as the shaded region in Figure 2.2

Activity 2.3

Draw two Venn diagrams, showing the following pairs of sets.

(a) $\{2, 1\}$ and $\{4, 3\}$. (b) $\{1, 2\}$ and $\{3, 2, 1\}$.

Discussion See Figure 2.3. In (a), the two sets have no element in common so the corresponding regions in the Venn diagram do not overlap. In (b), $\{1, 2\}$ is a subset of $\{3, 2, 1\}$, so one region lies entirely inside the other.

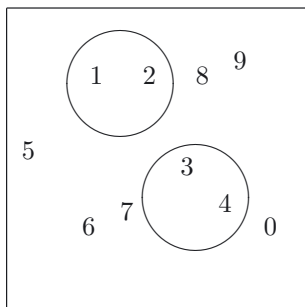
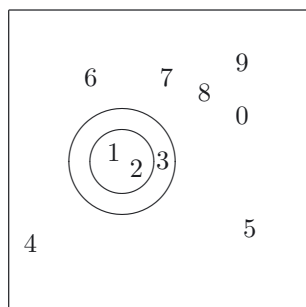


Figure 2.3 (a)



(b)

John Venn (1834–1923) was a fellow of Gonville & Caius College Cambridge from 1857 to 1923, where he studied and taught logic and probability theory. His invention of Venn diagrams is commemorated by a stained glass window at the college, designed by Maria McClafferty and installed in 1989. This can be viewed on the web at <http://www.cai.cam.ac.uk/rota.php?count=4>

2.3 Intersection of sets

The binary operation of **set intersection** forms a new set consisting of the elements common to two sets. It is written using the symbol $\cap$, so the intersection of sets A and B is written as $A \cap B$.

For example, the intersection of the sets $\{1, 2, 3, 4\}$ and $\{3, 4, 5\}$ is the set $\{3, 4\}$. Both sets contain the elements 3 and 4, and they have no other elements in common. We write

$$\{1, 2, 3, 4\} \cap \{3, 4, 5\} = \{3, 4\}.$$

Figure 2.4 shows the intersection $\{1, 2, 3, 4\} \cap \{3, 4, 5\}$ in a Venn diagram.

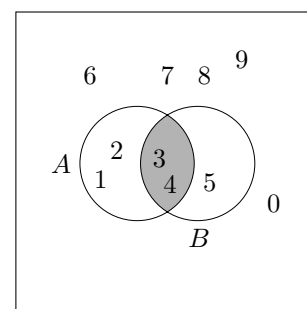


Figure 2.4 $A \cap B$ is shaded.

Activity 2.4

Enumerate each of the following sets and give its cardinality.

- $\{2, 1\} \cap \{4, 3\}$
- $\{1, 2\} \cap \{3, 2, 1\}$
- $\{\} \cap \{3, 2, 1\}$
- $F \cap G$, where F is the set of characters in the string “qwerty” and G is the set of characters in the string “quarter”.

Discussion

- The two sets have no element in common, so the intersection is the empty set $\{\}$. Such sets are said to be **disjoint**. The cardinality of this intersection is 0.
- This intersection is $\{1, 2\}$, with cardinality 2. Notice that $\{1, 2\}$ is a subset of $\{3, 2, 1\}$ and that in such a case the intersection is just the smaller set.
- This intersection is the empty set $\{\}$ (cardinality 0). The empty set has no elements, so cannot have any element in common with any other set.
- This intersection is the set $\{‘q’, ‘e’, ‘r’, ‘t’\}$, with cardinality 4. These four characters lie in both the sets F and G , and these sets have no other characters in common.

Notice that the cardinality of the intersection of two sets is always less than (or equal to) the cardinality of each of the two sets.

We can set out a “mechanisable” procedure for finding the intersection of two sets given by enumeration as below.

Strategy 2.2

To find the intersection $A \cap B$.

- Pick an element of the set A and check whether it is in the set B . If it is, then add it to the intersection.
- Cross out the element of A that you have examined.
- Repeat these two steps until all the items in A have been examined.

An element lies in the intersection of two sets precisely when it is an element of both sets. So we have the following characterisation of intersection, which does not depend on the sets being given by enumeration. Recall that the

symbol $\wedge$ is read as “and”, and this infix operation combines two Boolean values and returns **true** only if both of them are **true**.

Result 2.2

$x \in (A \cap B)$ has the same truth value as $(x \in A) \wedge (x \in B)$.

2.4 Difference of sets

The **difference** of the set A and the set B is the set consisting of the elements of A that are not in B . We shall write the binary operation of set difference using the infix symbol $-$. To form the set $A - B$, we remove from the set A any element that occurs in the set B .

The difference of the sets $\{1, 2, 3, 4\}$ and $\{3, 4, 5\}$ is the set $\{1, 2\}$. Note that the elements 3 and 4 of the first set are also in the second set, and these have been removed from the set $\{1, 2, 3, 4\}$ to form this set difference. So:

$$\{1, 2, 3, 4\} - \{3, 4, 5\} = \{1, 2\}.$$

Figure 2.5 shows the difference of these sets illustrated in a Venn diagram.

Not all texts use the symbol $-$ for set difference.

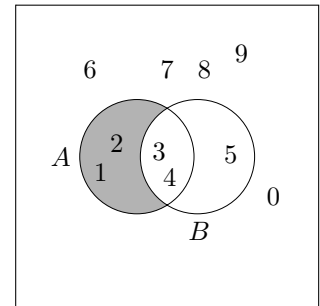


Figure 2.5 $A - B$ is shaded.

Activity 2.5

Enumerate each of the following sets. For (a), draw a Venn diagram showing the sets involved.

- $\{3, 4, 5\} - \{1, 2, 3, 4\}$
- $\{1, 2\} - \{3, 2, 1\}$
- $\{2, 1\} - \{4, 3\}$
- $\{3, 2, 1\} - \{\}$
- $\{2, 0, 1, 5, 8, 6, 7, 3\} - \{0, 1, 2, 3\}$
- The difference $F - G$, where F is the set of characters in the string “qwerty” and G is the set of characters in the string “quarter”.

Discussion

- To form $\{3, 4, 5\} - \{1, 2, 3, 4\}$, remove from the set $\{3, 4, 5\}$ the elements 3 and 4 that also appear in $\{1, 2, 3, 4\}$. So the difference of these sets is $\{5\}$. The sets are illustrated in a Venn diagram in Figure 2.6.
- $\{1, 2\} - \{3, 2, 1\} = \{\}$. Each of the items in $\{1, 2\}$ is also in $\{3, 2, 1\}$.
- $\{2, 1\} - \{4, 3\} = \{2, 1\}$. The sets $\{2, 1\}$ and $\{3, 4\}$ have no elements in common (these sets are disjoint). So to form the set difference we remove nothing from the first set. (We also have $\{4, 3\} - \{1, 2\} = \{4, 3\}$.)
- There is nothing in the empty set, so nothing needs to be removed to form the set difference. We have $\{3, 2, 1\} - \{\} = \{3, 2, 1\}$.
- In this case, all the items in the second set $\{0, 1, 2, 3\}$ are also in the first set $\{2, 0, 1, 5, 8, 6, 7, 3\}$. So to form the difference, we remove all of 0, 1, 2 and 3 from $\{2, 0, 1, 5, 8, 6, 7, 3\}$, to obtain the set $\{5, 8, 6, 7\}$.
- $F - G = \{'w', 'y'\}$.

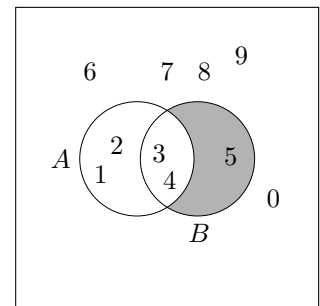


Figure 2.6 $B - A$ is shaded.

A “mechanisable” procedure for finding the difference of two sets given by enumeration is given in Strategy 2.3.

Strategy 2.3

To find the set difference $A - B$.

- Choose an element of A and check whether it appears in B . If it does not, then add it to the set difference.
- Cross out that item from A .
- Repeat these two steps for each item in A .

An element lies in the difference of two sets precisely when it is an element of the first but not the second. So we have the following characterisation of intersection, which does not depend on the sets being given by enumeration.

Result 2.3

$x \in (A - B)$ has the same truth value as $(x \in A) \wedge (\text{NOT}(x \in B))$.

Figure 2.7 shows a Venn diagram for the sets in Activity 2.5(e). Notice that $\{0, 1, 2, 3\}$ is a subset of $\{2, 0, 1, 5, 8, 6, 7, 3\}$. Where B is a subset of A , we refer to the difference $A - B$ as the **relative complement** of the set B in the set A .

If A is a set drawn from a predefined set U , then $U - A$ is called the **complement** of A . If you read mathematics texts about sets, the idea of complement will be freely used. If both A and B are given by enumeration, then we can readily give the difference $A - B$ by enumeration. If we are working with integers drawn from the predefined set $SmInt$, which can be enumerated itself, then there is no problem with enumerating the complement of a set. However, consider sets of integers drawn from the predefined set of all implemented integers, $ImpInt$. A set such as $ImpInt - \{0, 1, 2, 3\}$ certainly cannot be given by enumeration on paper, and even to enumerate it on a machine would be impracticable. We emphasise the idea of set difference rather than complement for this reason.

Aside

In this unit, the idea of set plays two related but slightly different roles. Our main concern is with the idea of a set as a collection of items of a particular type. The data set underlying a type, such as $ImpInt$, gives the “universe” of all possible states of a variable of that type. We have referred to sets such as $ImpInt$ as predefined sets. In the foundations of the theory of sets, we need to separate the notions of a predefined set and a set that is a collection drawn from it. This is not something that will concern us in this unit. But if the roles are not separated, we can end up giving a description of a set such as this. “ X is the set whose members are all the sets that are not members of themselves.” Such a description contains an internal contraction (a paradox). A similar paradox is contained in the following description:

A barber living in Seville shaves everyone in the city who do not shave themselves, and no-one else.

So who shaves the barber?

End of Aside

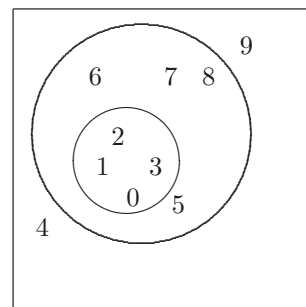


Figure 2.7

This is known as Russell’s paradox. For more, look under Bertrand Russell at the St Andrews University web site cited on p. 5.

Objectives for Section 2

After studying this section you should be able to do the following.

- ☐ Recognise and use the terminology: union; intersection; difference (of sets); relative complement; complement.
- ☐ For two sets given by enumeration, give their union, intersection and difference.
- ☐ Draw Venn diagrams to visualise these concepts.

Exercises on Section 2

Exercise 2.1

Let $A = \{1, 2, 3, 4, 7\}$ and $B = \{3, 5, 6\}$. Write down each of:

- (a) $A \cup B$; (b) $A \cap B$; (c) $A - B$.

Exercise 2.2

Let E be the set of characters appearing in the string “sentence” and F be the set of characters appearing in the string “length”.

- (a) Enumerate each of the sets: (i) $E \cup F$; (ii) $E \cap F$; (iii) $E - F$.
 (b) Give the cardinality of each of the sets: E ; F ; $E \cup F$; $E \cap F$.

Exercise 2.3

Let A , B and C be the sets $A = \{1, 2, 4, 5, 6\}$, $B = \{0, 4, 5, 9\}$, $C = \{0, 4, 6, 7\}$.

- (a) Enumerate each of the sets: (i) $A - (B \cup C)$, (ii) $(A - B) \cap (A - C)$.
 (b) Draw a Venn diagram illustrating the set in (a) (i).
 (c) Suppose that A , B and C are any three sets containing items of the same type, and x is an item of that type. Relate the proposition $x \in (A - (B \cup C))$ to a combination of the propositions $x \in A$, $x \in B$ and $x \in C$.

3 The class Set

Consider a monochrome (say black and white) computer screen. We might visualise this as a rectangle of locations (called **pixels**) each of which is either “on” or “off”. A graphic shown on the screen corresponds to a set of pixels: those that are “on”. In manipulating such graphics the operations on sets can be useful, and this is a situation where a class that makes available methods implementing the various processes on sets described in Sections 1 and 2 might be used in practice.

In this section, we shall describe an example that you can think of as a computer screen, although one that is very much “scaled down”, so that we can give sets by enumeration. Our model screen consists of just 16 pixels, arranged in a four-by-four grid. Figure 3.1 shows: (a) these sixteen locations; and (b) an example with a particular set of locations marked.

In practice, a computer screen will contain many more pixels. A screen display showing 1152 by 862 pixels will have a total of $1152 \times 862 = 993024$ locations. In such a context, sets given by enumeration may be very large!

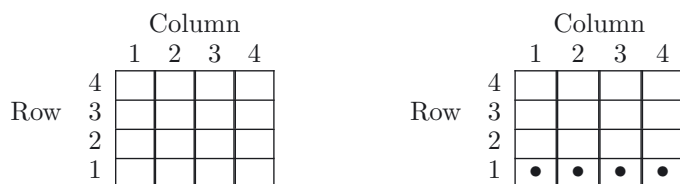


Figure 3.1 (a) A grid of 16 locations. (b) A particular set of locations.

To be able to use processes on sets to manipulate such figures, we would want a class with two key features.

- The underlying data set is the collection of all sets drawn from the grid of locations shown in Figure 3.1(a).
- It has methods that permit one to perform the various processes on sets discussed in Sections 1 and 2.

Later in this section, we will specify a class **Set of X** which makes available processes on sets. That class has as its underlying data set *SetOfX*, which contains all sets drawn from *X*. Before specifying the class **Set of X**, we will look first at two more ideas associated with sets in general that were briefly mentioned in Unit 1. These are the power set, and the Cartesian product of two sets. The Cartesian product helps us to describe a set that can be used to represent the grid shown in Figure 3.1(a).

Set of X is a parameterised class.

3.1 The power set

Suppose that the variable *mySet* is of type **Set of Char**. Then the state of *mySet* will be a set of characters. The data set of the class **Set of Char** consists of all sets of characters. In Unit 1, we denoted this set by *SetOfChar*. This is called the **power set** of the set *Char*.

Our main interest in the idea of power set relates to a situation where we consider all sets drawn from a predefined set, such as *Char* or *ImpInt*. If **X** is some type, with data set *X*, say, then the data set of the type **Set of X** will be the power set of *X*, *SetOfX*.

However, the idea of power set can be applied to any set. For a small set given by enumeration, we can also give the power set by enumeration. For example, the power set of the set $\{4, 6, 1\}$ is enumerated below.

$\{\{\}, \{4\}, \{6\}, \{1\}, \{4, 6\}, \{4, 1\}, \{6, 1\}, \{4, 6, 1\}\}$

Notice that we include in the power set of *X* both the set *X* itself and the empty set. Notice also that, in this example, $\{4, 6, 1\}$ has cardinality 3, while its power set has cardinality 8, which is 2^3 . This result about cardinality can be generalised. If *X* has cardinality *n*, then the cardinality of its power set is 2^n .

This is the source of the use of the word “power” in the name power set.

Activity 3.1

For each of the following sets, enumerate its power set, and also give its cardinality.

- $\{1, 2\}$
- The set of characters in the string “abc”.
- $\{\}$

Discussion

- (a) This power set is $\{\{\}, \{1\}, \{2\}, \{1, 2\}\}$. This has cardinality 4 ($= 2^2$).
- (b) This power set is $\{\{\}, \{a\}, \{b\}, \{c\}, \{a, b\}, \{a, c\}, \{b, c\}, \{a, b, c\}\}$. This has cardinality 8 ($= 2^3$).
- (c) The only subset of $\{\}$ is itself, hence the power set is the set $\{\{\}\}$. (This notation shows a set with one member, which is the empty set.) The cardinality of this power set is 1 ($= 2^0$).

The size of the power set grows rapidly as the size of the set increases. So it rapidly becomes impractical to give power sets by enumeration, particularly by hand. For example, for a set of cardinality 16 (as in our grid example), the power set has cardinality $2^{16} = 65536$.

3.2 Cartesian product of sets

We briefly discussed the Cartesian product of sets in Unit 1. Given two sets X and Y , their **Cartesian product**, denoted by $X \times Y$, consists of all **ordered pairs** (x, y) , where x is from X and y is from Y . Here, the word “ordered” emphasises that the pairs (x, y) and (y, x) are regarded as different. For example, if you think about an object of type **Spot**, the states (1,3) and (3,1) represent different locations. Each of (1,3) and (3,1) is an ordered pair of integers, from the set $Int \times Int$.

In (x, y) :

- x is called the **first component** of the pair;
- y is called the **second component** of the pair.

Two ordered pairs are equal only if their first components are the same and their second components are the same. Example 3.1 emphasises this point.

EXAMPLE 3.1

For the integers 1 and 3, the ordered pairs (1, 3) and (3, 1) are different.

Cartesian products are mainly of interest as a way of forming predefined sets, as we shall discuss shortly. But, as with power sets, the idea is applicable to any two sets. The cardinality of any Cartesian product set, $X \times Y$, is found by multiplying together the cardinalities of X and of Y . If we have two sets given by enumeration, then we can enumerate their Cartesian product using the procedure below.

Strategy 3.1

To find the Cartesian product $A \times B$.

Form a list of ordered pairs as follows.

- Choose an element, say a , of A . Place a in the first position of an ordered pair. Now work through the set B , forming all the possible pairs with a as the first component and an element of B as the second component.
- Repeat the above for every other element of A .

Example 3.2 illustrates this procedure.

If $A = \{1, 2\}$ and $B = \{1, 2, 3\}$ then we first form the ordered pairs $(1, 1)$, $(1, 2)$, $(1, 3)$ and then the pairs $(2, 1)$, $(2, 2)$, $(2, 3)$. The Cartesian product is then given by the set

$$A \times B = \{(1, 1), (1, 2), (1, 3), (2, 1), (2, 2), (2, 3)\}.$$

Whether or not the sets A and B are enumerated, we can recognise a member of any Cartesian product using the following test.

Result 3.1

For any elements x and y of the appropriate types, the proposition $(x, y) \in A \times B$ has the same truth value as the composite proposition $(x \in A) \wedge (y \in B)$.

Activity 3.2

- (a) Enumerate each of the Cartesian product sets:
 - (i) $\{2, 1\} \times \{4, 3\}$; (ii) $\{\} \times \{3, 2, 1\}$.
- (b) (i) What is the cardinality of $SmInt \times \{0, 1, 2\}$?
 (ii) Which of the following lies in this set: $(11, 0)$; $(2, 8)$, $(9, 0)$; 4?

Discussion

- (a) (i) Using Strategy 3.1 above, we get $\{(2, 4), (2, 3), (1, 4), (1, 3)\}$.
 (ii) There are no elements in $\{\}$, and so no ordered pairs can be formed. Hence the resulting Cartesian product is an empty set.
- (b) (i) This set contains $10 \times 3 = 30$ elements, since $SmInt$ has cardinality 10 and $\{0, 1, 2\}$ has cardinality 3 (not really the sort of set we want to give by enumeration in text). (ii) A pair of integers (x, y) will be in this Cartesian product if x is in $SmInt$ and y is in $\{0, 1, 2\}$. Of the given items, only $(9, 0)$ is in this Cartesian product. 11 is not in $SmInt$; 8 is not in the set $\{0, 1, 2\}$; while 4 is an integer, not a pair of integers.

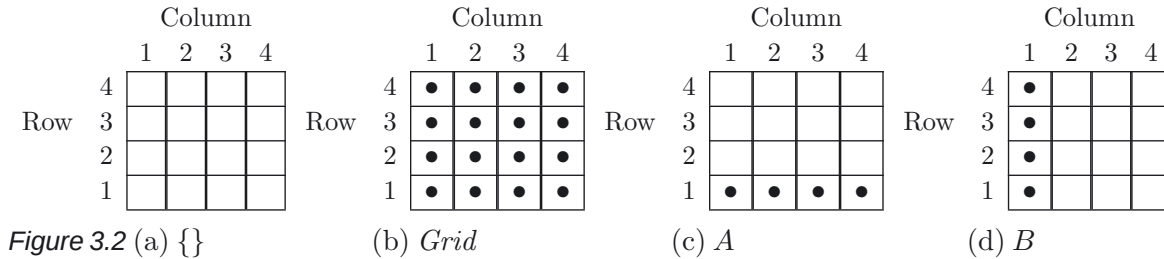
3.3 The set *Grid*

In Figure 3.1 we illustrated a four-by-four grid that models a “scaled down” computer screen. We can regard this grid as a Cartesian product. The set of possible locations on the grid can be represented by $\{1, 2, 3, 4\} \times \{1, 2, 3, 4\}$. We will call this set *Grid*. In an ordered pair in *Grid*, the first integer will represent the column in the grid, and the second integer the row. So, for example, the top-left location on the grid (column 1, row 4) is represented by the ordered pair (1,4).

We will denote by *Image* the set consisting of all sets drawn from *Grid*. (So *Image* is the same as *SetOfGrid*.) Then each member of *Image* is a set of ordered pairs. The cardinality of *Grid* is $4 \times 4 = 16$, and so the cardinality of *Image* is $2^{16} = 65536$.

For example, Figure 3.2(c) shows the set $\{(1,1), (2,1), (3,1), (4,1)\}$ from *Image*. Each square on the grid corresponding to an ordered pair in the set is marked in the figure by a •, the other squares being left unmarked.

Figure 3.2 shows four particular sets from *Image*: (a) the empty set $\{\}$; (b) the whole set *Grid*, (c) the set $A = \{(1,1), (2,1), (3,1), (4,1)\}$; and (d) the set $B = \{(1,1), (1,2), (1,3), (1,4)\}$.



Activity 3.3

Let A and B be as defined above, and illustrated in Figure 3.2(c) and (d).

- (a) Draw the diagrams for the following elements of *Image*: (i) $A \cup B$; (ii) $A \cap B$; (iii) $A - B$; (iv) $B - A$; (v) $Grid - A$; (vi) $Grid - B$.
 (b) Give the element of *Image* in (a) (iii) as a set of pairs.

Discussion (a) The diagrams are shown in Figure 3.3.

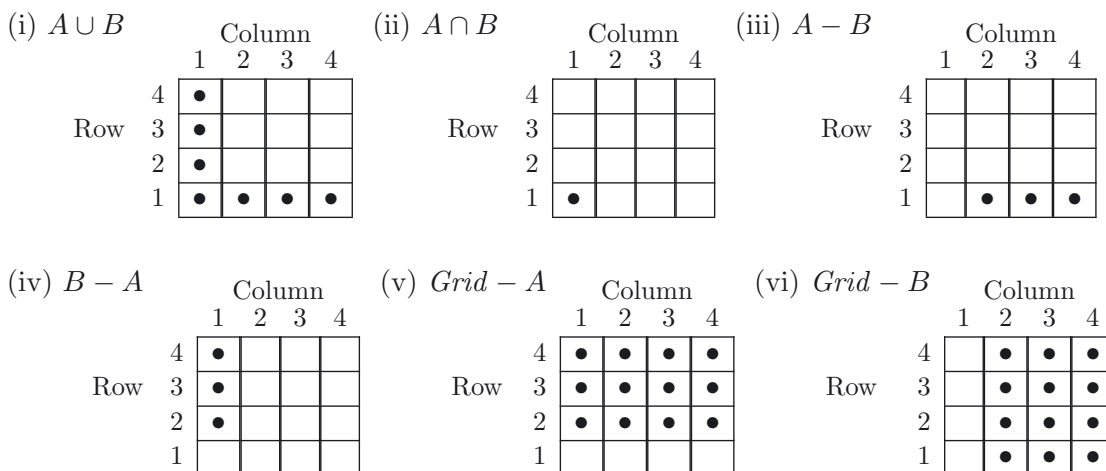


Figure 3.3

(b) As a set of pairs, $A - B = \{(2,1), (3,1), (4,1)\}$.

For any set A in *Image*, the relative complement $Grid - A$ is formed by “reversing” every square in the grid, so that squares containing $\bullet$ are changed to empty squares, while empty squares are changed to contain $\bullet$.

Two results about relative complements in *Grid* are stated below. These results actually apply to all sets, and are known as **De Morgan’s Laws**. We shall meet these general results later in the course. For now, satisfy yourself that they work for the sets A and B given in Activity 3.3. Notice how the intersection of the sets shown in Figure 3.3(v) and (vi) corresponds to the unmarked squares in Figure 3.3(i). Then compare the union of the sets in Figure 3.3(v) and (vi) with that in Figure 3.3(ii).

Indeed, there are a number of results about how the various operations on sets interact.

Result 3.2

$$Grid - (A \cup B) = (Grid - A) \cap (Grid - B).$$

$$Grid - (A \cap B) = (Grid - A) \cup (Grid - B).$$

Aside

Modelling of images on a computer screen by sets is used in practice. For example, a technique called mathematical morphology is used for pre-processing graphical images to fill in holes and remove noise. This technique is based on set theory. It involves two operations called dilation and erosion. We will give a description of these operations in a simple case. The predefined set is a set of points in the plane that we shall call *Pixels*. Let r be a fixed small number (i.e. a real number close to 0). If p is an element of *Pixels*, the set B_p is defined to be the set of points in *Pixels* that are within a distance r of p ; that is, the points of B_p lie in a small disc whose centre is p and whose radius is r . For a set A drawn from *Pixels*:

the **dilation** of A by r is the set $\{p \text{ in } Pixels : B_p \cap A \neq \{\}\}$;

the **erosion** of A by r is the set $\{p \text{ in } Pixels : B_p \subseteq A\}$.

These operations together with set union and intersection form the basis of the technique. We do not expect you to be able easily to picture what these operations do nor how they help in image processing. But notice how they are based on processes on sets that we have introduced in this unit.

To see some simple examples visit
www.cis.rit.edu/class/simg782.old/lec_morphology.html

End of Aside

3.4 Classes of pairs

In Subsection 3.5, we shall introduce a parameterised class **Set of X** in which various operations on sets are available. To use that class in the context of the sort of figures we were looking at in the previous subsection, we would want to use a class **Set of Grid**, say, where the class **Grid** has a data set that is the set *Grid*.

Now the set *Grid* consists of pairs of integers. Rather than specify a type for the specific application, we look first at a very general class of pairs. We can readily specify and implement a class whose data set consists of pairs. A parameterised class **Pair** is specified below.

The class **Pair** is in the WorkPad library.

Pair of X,Y	Class specification
<i>States</i>	The state of an object is an ordered pair of elements (x, y) , where x is in X and y is in Y .
<i>State notation</i>	(x, y) .
<i>Default</i>	There is no default.
<i>Methods</i>	
	<i>setPair</i> (x in X , y in Y) state change

method *setPair*(x in *X*, y in *Y*) state change

pre **true**.

post The state of the receiver is set to (x, y) .

As there is no default state, a variable of type **Pair** must be initialised using the method *setPair*.

The class **Pair** can be implemented using two attributes, one of type **X** and one of type **Y**. If **X** has data set *X* and **Y** has data set *Y*, then the data set of **Pair of X,Y** is the Cartesian product $X \times Y$. Notice how the components in the ordered pair in the Cartesian product correspond to the attributes in the implementation. There is a close relationship between Cartesian products and the data sets of classes. In essence, the data set of any class is the Cartesian product of the data sets of each of its attributes. This correspondence extends to implementations with more than two attributes, where the corresponding Cartesian product will involve several sets.

The details of the implementation of **Pair** are straightforward, and are omitted.

In the next activity, we see how we might specify and implement a class **GridPair** whose data set has the values of its two integer attributes restricted to the values 1, 2, 3 or 4.

Activity 3.4

A class **GridPair** is specified below. Give an implementation of **GridPair** in a form suitable for the WorkPad.

This activity is a reminder of how a user-defined class may be coded in the WorkPad. The class **GridPair** has no significance in your later work!

GridPair	Class specification
<i>States</i> The state of an object is an ordered pair of integers representing a location in a four-by-four grid.	
<i>State notation</i> (x, y) , where x gives the column and y gives the row.	
<i>Default</i> (1,1)	
<i>Methods</i>	
<i>setPix</i> (x in Int, y in Int) state change	
<i>getCol</i> () return in Int	
<i>getRow</i> () return in Int	

method *setPix*(x in Int, y in Int) state change

pre true.

post If $1 \leq x \leq 4$ and $1 \leq y \leq 4$ then the state of the receiver is set to (x, y) . Otherwise the receiver is unchanged.

method *getCol*() return in Int

pre true.

post If the state of the receiver is (x, y) then the returned value is x .

method *getRow*() return in Int

pre true.

post If the state of the receiver is (x, y) then the returned value is y .

The postcondition for *setPix* (together with the default value) ensures that the state (x, y) of an object of this class always has the values of x and y confined to the range 1 to 4.

Discussion To implement **GridPair**, we can use two attributes of type **Int** (each of whose values must lie between 1 and 4). An implementation of **GridPair** in a form suitable for the WorkPad is given below. Statements (1) and (2) set the default value for this class. (3) is required by any class in the WorkPad. The **if** condition in *setPixGridPair* ensures that the state is only set to (x, y) if the values of x and y lie in the required ranges. Otherwise, the state of the receiver will not be changed.

```

Class GridPair

{
  this.col in Int
  this.row in Int
  this.col <-- 1                //(1)
  this.row <-- 1                //(2)
  this.setPix calls setPixGridPair
  this.getRow calls getRowGridPair
  this.getCol calls getColGridPair
}

GridPair extends Base          //(3)

```

```
method setPixGridPair(x,y)
{
if ((1 <= x) /\ (x <= 4) /\ (1 <= y) /\(y <= 4)) then
{
    this.col <-- x
    this.row <-- y
}
}
```

```
method getRowGridPair()
{
return this.row
}
```

```
method  getColGridPair()
{
return this.col
}
```

The class **GridPair** is not in the WorkPad library.

3.5 The class Set

We now specify a class that has methods corresponding to various operations on sets described in Sections 1 and 2. This is a parameterised class: the type of objects that a set contains is given by the parameter **X**. We will not give details of how the class is implemented, but note that the implementation is based on sets given by enumeration. Implementations of the methods can be based on the strategies described earlier for executing the various operations when the sets are given by enumeration.

The specification of the class **Set of X** is given below.

Set of X	Class specification
<i>States</i>	The state of an object is a set of elements of type X .
<i>State notation</i>	A list of elements enclosed in curly brackets and separated by commas.
<i>Default</i>	The empty set {}.
<i>Methods</i>	
	<i>setSet(list of X)</i> state change
	<i>addElement(x in X)</i> state change
	<i>card()</i> return in Int
	<i>contains(element in X)</i> return in Bool
	<i>containsSet(A in Set of X)</i> return in Bool
	<i>equals(A in Set of X)</i> return in Bool
	<i>union(A in Set of X)</i> return in Set of X
	<i>intersect(A in Set of X)</i> return in Set of X
	<i>minus(A in Set of X)</i> return in Set of X
	<i>cartProd(A in Set of Y)</i> return in Set of Pair of X,Y

The first method, *setSet*, is similar to the method *setVector* of the class **Vector**. The parameter *list* is a list of items, each of type **X**. The resulting state is the set derived from the list by enumeration; that is, any repeated values in the list are ignored. For example, if *s* is of type **Set of Int**, the statement *s.setSet(2,1,3,3,1,2)* would change the state of the object *s* to {2,1,3}.

method *setSet(list of X)* **state change**

pre **true**.

post The state of the receiver becomes the set derived from *list* by enumeration.

The next method changes the state of an object by adding an element to it.

method *addElement(x in X)* **state change**

pre **true**.

post The element *x* is added to the receiver (if it is not already there).

The two methods above are the only state-changing methods of the class **Set**. The other methods return values, each of which is obtained by a process on sets discussed earlier. The returned values are summarised below, where *R* denotes the receiver set. A full specification of each method is then given.

method	returned value
<i>card</i>	The cardinality of <i>R</i>
<i>contains(element)</i>	$element \in R$
<i>containsSet(A)</i>	$A \subseteq R$
<i>equals(A)</i>	$R == A$
<i>union(A)</i>	$R \cup A$
<i>intersect(A)</i>	$R \cap A$
<i>minus(A)</i>	$R - A$
<i>cartProd(A)</i>	$R \times A$

method *card()* **return in Int**

pre **true**.

post The returned value is the number of elements in *R* (its cardinality).

In these method specifications, *R* is used to denote the receiver.

method *contains(element in X)* **return in Bool**

pre **true**.

post The returned value is **true** if *R* contains *element* as a member, and is **false** otherwise.

method *containsSet(A in Set of X)* **return in Bool**

pre **true**.

post The returned value is **true** if *A* is a subset of *R*, and is **false** otherwise.

method *equals(A in Set of X)* **return in Bool**

pre **true**.

post The returned value is **true** if *A* and *R* are equal as sets, and is **false** otherwise.

method *union(A in Set of X)* **return in Set of X**

pre **true**.

post The returned value is the set $R \cup A$.

method *intersect(A in Set of X)* **return in Set of X**

pre **true**.

post The returned value is the set $R \cap A$.

method *minus*(*A* in Set of X) return in Set of X

pre true.

post The returned value is the set $R - A$.

method *cartProd*(*A* in Set of Y) return in Set of Pair of X,Y

pre true.

post The returned value is the set $R \times A$.

Activity 3.5

What is the state of *A* after the code below has been executed?

```
var A in Set of Int
A.setSet(2,3,1,1,1,2,3)      //(1)
A.addElement(99)             //(2)
```

Discussion After (1) the state of *A* will be $\{2, 3, 1\}$, and after (2) the state will be $\{2, 3, 1, 99\}$.

If you wrote down the elements in a different order, then this is also correct.

Activity 3.6

Suppose that *A* and *B* are of type **Set of Int**, and are in the states:
A, $\{3, 2, 4, 1\}$ and *B*, $\{2, 3, 4\}$.

(a) Write down the values returned by each of the following method calls:

- (i) *A*.card(); (ii) *A*.contains(3); (iii) *A*.contains(5);
(iv) *A*.containsSet(*B*); (v) *B*.containsSet(*A*); (vi) *A*.equals(*B*).

(b) Suppose that the statement *B*.addElement(1) is then executed. What value is now returned by *A*.equals(*B*)?

Discussion

(a) The returned values are as follows:

- (i) 4; (ii) **true**; (iii) **false**; (iv) **true**; (v) **false**; (vi) **false**.

(b) After the statement *B*.addElement(1), the state of *B* will be $\{2, 3, 4, 1\}$. The value returned by *A*.equals(*B*) will then be **true**.

Activity 3.7

Suppose that *A* and *B* are of type **Set of Int**, and are in the states:
A, $\{3, 2, 4, 1, 7\}$ and *B*, $\{2, 3, 4\}$.

Write down the values returned by each of the following method calls:

- (a) *A*.union(*B*); (b) *A*.intersect(*B*); (c) *A*.minus(*B*); (d) *A*.cartProd(*B*).

Discussion The returned values are given below. (If you have the same elements in a different order then this is also correct.)

- (a) $\{3, 2, 4, 1, 7\}$. (b) $\{3, 2, 4\}$. (c) $\{1, 7\}$.
(d) $\{(3, 2), (3, 3), (3, 4), (2, 2), (2, 3), (2, 4), (4, 2), (4, 3), (4, 4), (1, 2), (1, 3), (1, 4), (7, 2), (7, 3), (7, 4)\}$.

Activity 3.8

Give the state of A after the following code has been executed.

```
var p1 in Pair of Int, Int
var p2 in Pair of Int, Int
p1.setPair(3, 1)
p2.setPair(4, 3)
var A in Set of Pair of Int, Int
A.setSet(p1, p2)
```

Discussion The state will be a set of pairs: $\{(3, 1), (4, 3)\}$. It would be correct to write this as $\{(4, 3), (3, 1)\}$, but the order of the items in each pair must be correct. $\{(3, 1), (3, 4)\}$ is wrong!

Activity 3.9

(a) What is the state of A after the following code has been executed?

```
var X in Set of Int
var Y in Set of Int
var A in Set of Pair of Int, Int
X.setSet(1, 2, 3, 4)
Y.addElement(2)
A <-- X.cartProd(Y)
```

(b) The state in (a) is a subset of the set we called *Grid*. Sketch the state in the style used earlier to sketch subsets of *Grid*.

Discussion

(a) First the state of X is set to $\{1, 2, 3, 4\}$. After the **var** statement, Y will be in the default state, which is the empty set. The state of Y is then changed to $\{2\}$ by the statement $Y.addElement(2)$. The final statement then sets the state of A to: $\{(1, 2), (2, 2), (3, 2), (4, 2)\}$.

(b) The state in (a) is illustrated in Figure 3.4.

		Column			
		1	2	3	4
Row	4				
	3				
	2	•	•	•	•
	1				

Figure 3.4

Activity 3.9 involved a variable of type **Set of Pair of Int, Int**. For such a variable, the state will correspond to a set of items from *Grid* so long as each component in every pair lies in the range 1 to 4 (inclusive).

Objectives for Section 3

After studying this section you should be able to do the following.

- ☐ Recognise and use the terminology: power set; Cartesian product; ordered pair; component (of an ordered pair).
- ☐ Trace the effect of code involving methods of the classes **Set** and **Pair**.
- ☐ Given a set of items from *Grid*, sketch the diagram that represents it.

Exercises on Section 3

Exercise 3.1

Let $A = \{ 'a', 'b' \}$. Enumerate the power set of A and give its cardinality.

Exercise 3.2

Let A and B be the subsets of *Grid*: $A = \{(1,1), (2,2), (3,3), (4,4)\}$ and $B = \{(1,4), (2,3), (3,2), (4,1)\}$. Sketch each of the following sets.

(a) A . (b) B . (c) $(Grid - A) \cap (Grid - B)$.

Exercise 3.3

Suppose that A and B have been declared and initialised as in the following fragment of code.

```
var A in Set of Char
var B in Set of Char
A.setSet('a','c','t')
B.setSet('a','d','s')
```

Write down the values returned by each of the following method calls:

(a) $A.card()$; (b) $B.union(A)$; (c) $A.intersect(B)$; (d) $A.minus(B)$;
(e) $B.minus(A)$; (f) $A.cartProd(B)$.

Exercise 3.4

Suppose that the fragment of code below has been executed.

```
var X in Set of Int
var Y in Set of Int
var A in Set of Int
var B in Set of Int
var C in Set of Int
var aGrid in Set of Pair of Int,Int
var AxB in Set of Pair of Int,Int
var AxC in Set of Pair of Int,Int
X.setSet(1,2,3,4)
Y <-- X.clone()
aGrid <-- X.cartProd(Y)
A.setSet(3,4)
B.setSet(1,2)
C.setSet(4,5,6)
AxB <-- A.cartProd(B)
AxC <-- A.cartProd(C)
```

(a) The state of AxB is an element of *Image*. Sketch it.
(b) What values (if any) would be returned by each of the following?
(i) $aGrid.containsSet(AxB)$
(ii) $aGrid.contains(AxB)$
(iii) $aGrid.containsSet(AxC)$

4 Set comprehension

We have so far dealt with small sets that we can enumerate. We now consider a way to describe sets that does not require us to explicitly write down all the elements. In this approach, we describe the set by giving a condition that its members must satisfy. This approach is called **set comprehension**. For example, let *UnderFive* be the set of integers that are less than 5. We can write this set as

$$\text{UnderFive} = \{x \text{ in } \text{Int} : x < 5\}.$$

Fundamental to this approach is the use of conditions like $x < 5$, and we look at these first.

To be precise, our approach is **typed set comprehension**, because the first thing we do is to state the type of the members of the set.

4.1 Predicates

Consider the expression $x < 5$, where the variable x is an integer. This expression asserts that x is less than 5. We separate out the phrase “less than 5” (written formally as < 5) that is used to qualify x . This is called a **predicate**. If the variable x is given a specific integer value then the expression $x < 5$ becomes a proposition. For instance, replace the x in $x < 5$ by the specific integer 4. We then get the proposition $4 < 5$. This proposition has the truth value **true**. Alternatively, we can replace the x in $x < 5$ by some other integer value, such as 5. We then get a different proposition, $5 < 5$. This proposition is **false**.

A number of examples of predicates are given in Example 4.1. In particular, item 3 there shows that a predicate may relate to more than one variable. We first give a general definition of the term.

Suppose we have an expression containing one or more variables such that, when the variables are all replaced by specific values (of appropriate types), the whole expression is a proposition. We call the phrase that qualifies the variables a **predicate**. We call the whole expression a **predicate sentence**.

EXAMPLE 4.1

In each of the following predicate sentences, x is an integer, as is y in (3).

1. $x == 5$.

The predicate is $== 5$ and the predicate sentence has one variable. When x is given a specific integer value the resulting proposition has a truth value. For instance $4 == 5$ is **false**, while $5 == 5$ is **true**.

2. x is even.

The predicate is “is even”. The predicate sentence has one variable. Examples of propositions obtained by giving x a specific value in this expression would be: “7 is even”, which is **false**, and “6 is even”, which is **true**.

3. $x < y$

This predicate sentence has two variables. To obtain a proposition in such a case, we need to give specific values to both these variables. For example, if we replace y by 11 and x by 2, we obtain the proposition $2 < 11$. This is **true**.

4. $(x > 3) \wedge (x < 5)$.

This predicate sentence has one variable. The predicate is “greater than 3 and less than 5”. If x is replaced by the integer 4 then we have the composite proposition $(4 > 3) \wedge (4 < 5)$ which has value **true** $\wedge$ **true**; that is, **true**.

Let us summarise the differences between a proposition and a predicate.

- A proposition has a truth value (**true** or **false**).
- A predicate relates to one or more variables.
- A predicate sentence becomes a proposition when all the variables are replaced by an appropriate value.
- When the variables in a predicate sentence are replaced by specific values we obtain a family of propositions, each of which is either **true** or **false**.

Predicates have much in common with functions. It is common practice in mathematics to use phrases such as: the function x^2 . This is a convenient abuse of language. Strictly, one should refer to a function with input set Int and output set Int , which returns the value x^2 when the input is x . To be so precise is often clumsy, though, and so we may incorporate the input variable to which the function is applied into the description of the function, and deduce its type by implication.

It is convenient to use similar language when discussing predicates. For example, we will refer to the predicate $x > 5$ rather than to (say) “the predicate > 5 relating to an integer variable”. That is, we will not usually differentiate between a predicate and its related predicate sentence.

Activity 4.1

For each of the following predicates, write down the number of variables and determine the truth value of the proposition obtained when x is replaced by 4 and y by 1.

(a) $x == 2 + 3$. (b) $x == y + 3$. (c) $(x < 2) \vee (x > 7)$. (d) $x == x$.

Discussion

- (a) The predicate has one variable, x . The proposition $4 == 2 + 3$ is **false**.
- (b) Here there are two variables. (Both have to be replaced to get a proposition.) The proposition $4 == 1 + 3$ is **true**.
- (c) This is a predicate in one variable. Replacing x by 4 gives the composite proposition $(4 < 2) \vee (4 > 7)$, which has truth value **false** $\vee$ **false**; that is, **false**.
- (d) This predicate has one variable. Replacing x by 4 gives the proposition $4 == 4$ with value **true**. (It is tempting to say that $x == x$ is **true**, but note that this is a predicate sentence and can only be given a truth value when x is given a specific value.)

4.2 Set comprehension

Consider the following notation for a set.

$$\{c \text{ in } Char : ASC(c) > ASC('A')\} \quad (4.1)$$

The notation inside the brackets is in two parts. The part after the colon is a predicate and the part before the colon identifies any variables in the predicate and gives their type. The curly brackets themselves indicate that all the elements are collected into a single entity, a set. Thus in this case, the set consists of all members of *Char* whose ASCII codes are larger than that of the character 'A'.

We use the notation $c \text{ in } Char$ rather than $c \in Char$ here to emphasise that we are declaring what sort of thing c is. This is similar to a code statement declaring the type of a variable, where we write **var** $c \text{ in } Char$. We reserve the symbol $\in$ for use in propositions (and predicates); that is, for expressions of the form $x \in A$, which will be either **true** or **false** when we give a specific set A and a specific item x .

The general form of definition of a set A by comprehension is given below.

$$A = \{x \text{ in } X : expression(x)\} \quad (4.2)$$

Then A is the set of all the members of X for which the predicate $expression(x)$ evaluates to **true**.

Here, X must be a set that has already been defined, and so is referred to as the **predefined set**. The notation can be interpreted with any set X for which a definition has already been given. Usually, the predefined set will be the data set of some data type (either a primitive type or class) that we have described earlier in the course. In either case we refer to $x \text{ in } X$ as **typing**. If X is the data set of a type, then we can tie this typing to the typing of a variable in code.

After the colon in Equation 4.2 there is a predicate in one variable. This is known as the **selector predicate**. By typing the variable we ensure that we only give it suitable values in the selector predicate. For example, in Equation 4.1, the selector predicate is $ASC(c) > ASC('A')$. The typing of c constrains us to try only values from the set *Char* in place of c , and this ensures that we will then get a well defined proposition. If, for example, we were to try an integer such as 101 in place of c in $ASC(c) > ASC('A')$, then we would get an expression that is not type consistent. We cannot assign a truth value of either **true** or **false** in such a case.

Not all texts use this notation for set comprehension. Mathematics texts often use $\in$ rather than **in**, while many computing texts use the notation $\{x : X | expression(x)\}$.

EXAMPLE 4.2

Consider the set

$$K = \{x \text{ in } SmInt : x < 5\}.$$

In this case the predefined set is *SmInt*. As this is small, we can enumerate all the propositions that we get by replacing x by each value in *SmInt*. Doing this gives 10 propositions. These are:

$$0 < 5; 1 < 5; 2 < 5; 3 < 5; 4 < 5; 5 < 5; 6 < 5; 7 < 5; 8 < 5; 9 < 5.$$

Of these, the first 5 are **true** and the remainder are **false**. In this case we can enumerate the set K , to get

$$K = \{x \text{ in } SmInt : x < 5\} = \{0, 1, 2, 3, 4\}.$$

We could use code to collect together the members of the set K as below. This uses the fact that we can work through the candidate values – the members of the set *SmInt* – by using a **for** loop to iterate through the integers from 0 to 9. In statement (1), the selector predicate becomes a proposition for each candidate value of x . If this proposition is **true**, then that value of x is added to the set enumeration by (2).

```
var A in Set of Int
var x in Int
for (x <-- 0 to 9) //iterate through the elements of SmInt
{
  if (x < 5) then           // (1)
  {
    A.addElement(x)         // (2)
  }
}
```

Activity 4.2

For each of the sets below, write down the predefined set and the selector predicate. Where possible, give the set by enumeration. Where this is not possible, give a value that belongs to the set and a value from the predefined set which does not belong to the set.

- (a) $A = \{x \text{ in } SmInt : x > 5\}$
- (b) $B = \{x \text{ in } Int : (x > -2) \wedge (x < 3)\}$
- (c) $C = \{x \text{ in } Int : (x > 3) \vee (x < -2)\}$
- (d) $D = \{x \text{ in } A : x == 3\}$, where A is the set of part (a) above.

Discussion

- (a) The predefined set is *SmInt*, and the selector predicate is $x > 5$. Replacing x in the selector predicate by the elements in *SmInt* yields a family of propositions including for example, $4 > 5$, which is **false**, and $6 > 5$, which is **true**. The propositions that are **true** have x replaced by 6, 7, 8, and 9. Hence the set enumeration for the set A is $\{6, 7, 8, 9\}$.
- (b) The selector predicate is $(x > -2) \wedge (x < 3)$, and the predefined set is *Int*. The only values in *Int* for which the selector predicate is **true** are $-1, 0, 1$ and 2 . So we can enumerate the set B , as $\{-1, 0, 1, 2\}$.
- (c) The selector predicate is $(x > 3) \vee (x < -2)$, and the predefined set is *Int*. In this case we cannot enumerate the set C , which contains nearly all of the members of *Int*. For example, replacing x by 32 gives the proposition $(32 > 3) \vee (32 < -2)$, which is **true** $\vee$ **false** = **true**. There are a few members of *Int* for which the selector predicate is **false**. For example, replacing x by 0 gives the proposition $(0 > 3) \vee (0 < -2)$, which is **false** $\vee$ **false** = **false**.
- (d) Here the predefined set is $\{6, 7, 8, 9\}$, as determined in part (a). The selector predicate is $x == 3$. All of the propositions $6 == 3$, $7 == 3$, $8 == 3$ and $9 == 3$ are **false**. Hence $D = \{\}$, the empty set.

Activity 4.3

Write a fragment of code that will enumerate in a variable *mySet* the members of the set $\{x \text{ in } LInt : (x > -4) \wedge (x < 2)\}$.

Discussion We can modify the fragment of code given in Example 4.2. We need this time to iterate through all the integers in the set *LInt*; that is, those from -100 to 100 , inclusive. We also need to use the appropriate selector predicate.

```
var mySet in Set of Int
var x in Int
for (x <-- -100 to 100)
{
  if ((x > -4) /\ (x < 2)) then
  {
    mySet.addElement(x)
  }
}
```

Let L be the set of integers between 0 and 10000, inclusive. We would not describe such a set by enumeration, but rather by comprehension, as below.

$$L = \{x \text{ in } Int : (0 \leq x) \wedge (x \leq 10000)\}.$$

In this example the selector predicate is a composite. Now each of the two predicates that make up the composite predicate also defines a set, say:

$$M = \{x \text{ in } Int : 0 \leq x\} \quad \text{and} \\ N = \{x \text{ in } Int : x \leq 10000\}.$$

Consider an integer x that is in the intersection $M \cap N$. Since x is in M , $0 \leq x$ must be **true**, while since x is in N , $x \leq 10000$ must be **true**. So $(0 \leq x) \wedge (x \leq 10000)$ must also be **true**; that is, x is in L . So if x is in $M \cap N$, then x is in L . We can give a similar argument to show that if x is in L , then x must be in $M \cap N$. So these sets are the same: the proposition $L == M \cap N$.

is **true**.

In Section 2, we gave characterisations of the various binary operations on sets that are not dependent on the sets being given by enumeration. These were expressed in terms of set membership, but can be rewritten as below. (The equality signs here mean that the sets are always the same; that is, any test of equality between sets of the specified forms will result in the value **true**.)

Let A and B be two sets from the predefined set X . Then

$$\begin{aligned} A \cup B &= \{x \text{ in } X : (x \in A) \vee (x \in B)\} \\ A \cap B &= \{x \text{ in } X : (x \in A) \wedge (x \in B)\} \\ A - B &= \{x \text{ in } X : (x \in A) \wedge (NOT(x \in B))\} \end{aligned}$$

These results can be applied to sets given by comprehension. In each case, this shows a relationship between the set operation and a corresponding composite selector predicate. Put simply, we can translate $\cup$ as “or”, $\cap$ as “and”, and $-$ as “and not”.

Result 4.1

Let $p(x)$ and $q(x)$ be predicates about the predefined set X . Then

$$\begin{aligned} \{x \text{ in } X : p(x)\} \cup \{x \text{ in } X : q(x)\} &= \{x \text{ in } X : p(x) \vee q(x)\} \\ \{x \text{ in } X : p(x)\} \cap \{x \text{ in } X : q(x)\} &= \{x \text{ in } X : p(x) \wedge q(x)\} \\ \{x \text{ in } X : p(x)\} - \{x \text{ in } X : q(x)\} &= \{x \text{ in } X : p(x) \wedge (NOT(q(x)))\} \end{aligned}$$

Activity 4.4

Write each of the following as a single set given by comprehension. Can these sets be enumerated?

- (a) $\{x \text{ in } Int : x > 3\} \cap \{x \text{ in } Int : x < 7\}$
- (b) $\{x \text{ in } Int : x > 3\} \cup \{x \text{ in } Int : x < 7\}$

Discussion

- (a) $\{x \text{ in } Int : (x > 3) \wedge (x < 7)\}$. The set can be enumerated as $\{4, 5, 6\}$.
- (b) $\{x \text{ in } Int : (x > 3) \vee (x < 7)\}$. If you try various values for x , then you will find that, whatever integer value you try, either $x > 3$, or $x < 7$, or both, will have the value **true**. Every integer is either greater than 3, or is less than 7, or both. So every integer is in this set; that is, the set is Int itself. This gives a compact way of expressing this set, but we cannot give it by enumeration!

4.3 Using set comprehension

The selector predicates we have considered so far have involved just one variable. They may involve more, if the predefined set is a Cartesian product.

EXAMPLE 4.3

Enumerate the set

$$\{(x, y) \text{ in } SmInt \times SmInt : (x + 6) < y\}.$$

The selector predicate involves two variables. To enumerate the set we just need to identify the values of x and y in $SmInt$ for which the predicate evaluates to **true**. The variable x must lie in $SmInt$, so may take any of the values 0, 1, 2, 3, 4, 5, 6, 7, 8 or 9. With $x = 0$, y may take any of the values 7, 8 or 9. With $x = 1$, y could be 8 or 9. With $x = 2$, y could only be 9. If $x \geq 3$ and y is in $SmInt$, then $x + 6 < y$ cannot be **true**. So we have now listed all the possibilities. We can enumerate this set as below.

$$\{(0, 7), (0, 8), (0, 9), (1, 8), (1, 9), (2, 9)\}$$

Activity 4.5

Draw the diagram for the following subset of *Grid*.

$$\{(x, y) \text{ in } Grid : x + y < 5\}$$

Discussion We are looking for values of x and y between 1 and 4 such that the predicate $x + y < 5$ yields a proposition that is **true**. If y is fixed at 1 then the predicate becomes $x + 1 < 5$. In this case x can take values 1, 2 or 3 to produce a proposition with value **true**. So the pairs (1, 1), (2, 1) and (3, 1) must be in the set. Now if y is fixed at 2, then we require $x + 2 < 5$. This is **true** if x is 1 or 2, so the pairs (1, 2), (2, 2) are also in the set. If y is fixed at 3 then we require $x + 3 < 5$, which is **true** only if x is 1. So the pair (1, 3) is also in the set. For y fixed at 4 we require $x + 4 < 5$, and there is no such x in $\{1, 2, 3, 4\}$.

Combining all these pairs we get the diagram in Figure 4.1.

		Column			
		1	2	3	4
Row	4				
	3	•			
	2	•	•		
	1	•	•	•	

Figure 4.1

Activity 4.6

Draw the diagram for the following subset of *Grid*.

$$\{(x, y) \text{ in } Grid : y < 3\}$$

Discussion The selector predicate here does not involve x . However, we know that x is in $\{1, 2, 3, 4\}$ for the definition of *Grid*. We can thus allow x to take any of these four values. For instance $(1, 1)$ is in the set since $1 < 3$ is true, while $(1, 3)$ is not in the set since $3 < 3$ is **false**. We see that for the propositions to be **true** the second component of (x, y) must take the value 1 or 2, while the first component can take any of the values 1, 2, 3 or 4. The set has the diagram in Figure 4.2.

		Column			
		1	2	3	4
Row	4				
	3				
	2	•	•	•	•
	1	•	•	•	•

Figure 4.2

Ideas from set theory are important when obtaining information from data bases. The following example illustrates how this can work. On a machine, we would need access to predicates corresponding to “is studying” and “is registered in”.

EXAMPLE 4.4

Let *Students* be the set of all currently registered Open University students. With this predefined set, we might define sets such as those below by comprehension.

$OnM263 = \{s \text{ in } Students: s \text{ is studying M263}\}$

$InR02 = \{s \text{ in } Students: s \text{ is registered in region 02}\}$

Then, for example, the intersection $OnM263 \cap InR02$ is the set

$\{s \text{ in } Students: s \text{ is studying M263 and } s \text{ is registered in region 02}\}.$

Activity 4.7

A database of holiday cottages in a particular area records the following information about the cottages.

- The maximum number that the cottage can sleep.
- Bookings made for the cottage.

The predefined set consists of all the cottages in the database and is called *Cotts*. The number that a cottage can sleep is obtained from a function *sleeps* that is defined as follows.

function *sleeps*(*cot* in *Cotts*) **return** in **Int**

pre **true**.

post The returned value is the maximum number that *cot* can sleep.

Information about the bookings can be found using a predicate *available*(*cot*, *w*) where *cot* is one of the cottages and *w* is an integer indicating the week. The proposition derived from this predicate evaluates to **true** when the cottage *cot* is available in the week whose number is replacing *w*.

- Give descriptions by comprehension of the following sets.
 - A*: the set of cottages that can sleep at least 4.
 - B*: the set of cottages that can sleep more than 6.
 - C*: the set of cottages available in week 17.
- Use set operations to give, in terms of *A*, *B* and *C*, the set of cottages that are available in week 17 and that can sleep four, five or six people.

Discussion

- (a) (i) $A = \{cot \text{ in } Cottts : sleeps(cot) \geq 4\};$
 (ii) $B = \{cot \text{ in } Cottts : sleeps(cot) > 6\};$
 (iii) $C = \{cot \text{ in } Cottts : available(cot, 17)\}.$
- (b) The set we require is $C \cap (A - B).$

Objectives for Section 4

After studying this section you should be able to do the following.

- ☐ Recognise and use the terminology: predicate, variable, set comprehension, predefined set.
- ☐ Express suitable specified sets of elements by comprehension.
- ☐ For a set given by comprehension, give its enumeration where feasible.
- ☐ For a set given by comprehension with a composite predicate, use set operations to express it in terms of sets given by comprehension with simpler predicates.
- ☐ In the context of a database, use given sets and predicates to express sets of interest.

Exercises on Section 4

Exercise 4.1

- (a) For the sets in (i) and (ii), give the predefined set and the selector predicate, and enumerate the set.
- (i) $\{x \text{ in } SmInt : x < 2\}$
 (ii) $\{c \text{ in } Char : c > 'z'\}$
- (b) Write the sets in (i) and (ii) as unions or intersections of sets with simpler predicates, and enumerate each set.
- (i) $\{x \text{ in } SmInt : (x < 3) \vee (x > 8)\}$
 (ii) $\{c \text{ in } Char : (c > 'R') \wedge (c < 'W')\}$

Exercise 4.2

Use set comprehension to express the subset of *Grid* shown in Figure 4.3.

Exercise 4.3

Write a fragment of code that will enumerate in a variable *mySet* the members of the set $\{(x, y) \text{ in } Grid : x - y == 1\}.$

		Column			
		1	2	3	4
Row	4				
	3				
	2	•	•		
	1	•	•		

Figure 4.3

Summary of Unit 7

1. A set is a collection of items in which neither repetition of an item nor the order in which items are given is of significance. All the members of a set are drawn from a predefined set, which contains all the items of the appropriate type to be candidates for membership of the set. In this course, the predefined set is most often the data set of some type such as **Int** or **Char**. Sets can be described by enumeration or by comprehension. A set is given by enumeration when all its elements are given explicitly. A set is given by comprehension when it is described using a condition.

We regard set membership as a proposition, which may be **true** or **false**. The proposition “ x is a member of the set A ” is denoted by $x \in A$. For a set given by enumeration, one determines the truth value of such a proposition by examining each item in the enumeration in turn, to see whether or not there is an item equal to x .

The set B is a subset of the set A , written $B \subseteq A$, if every item in B is also in A . The sets A and B are equal (written $A == B$) if they contain exactly the same items. The proposition $A == B$ always has the same truth value as $(A \subseteq B) \wedge (B \subseteq A)$. We only regard propositions involving sets as well formed if they are type consistent.

2. We described three binary operations on sets: union ($\cup$), intersection ($\cap$) and difference ($-$). For sets A and B , $A \cup B$ contains all the items in either A or B (or both); $A \cap B$ contains those items that are in both A and B , while $A - B$ contains those items in A but not in B .
3. The power set of a set A consists of all the sets that are subsets of A , including the empty set $\{\}$ and A itself. The Cartesian product $A \times B$ of sets A and B consists of all the ordered pairs (a, b) , where a is in A and b is in B . We described a parameterised class **Set of X**, which has methods corresponding to various processes on sets.
4. A predicate contains one or more variables and becomes a proposition when the variables are given values of the appropriate type. Sets described by comprehension use predicates. We noted that the set operations $\cup$ and $\cap$ correspond to the operations $\vee$ and $\wedge$ on Boolean values.

Solutions to the exercises

Solution 1.1

(a) $\{\text{'d'}, \text{'i'}, \text{'g'}, \text{'e'}, \text{'r'}, \text{'o'}\}$.

(b) (i) is **true**; the character 'o' does appear in the set Q . (ii) is **false**; the character 'Q' does not appear in the set Q . (Do not confuse the identifier of the set with the characters that it contains!) (iii) is not a proposition. "o" is a string while Q is a set of characters, so the types are not consistent.

Solution 1.2

(a) and (b) are both **true**. A proper subset is also a subset. Both (c) and (e) are **false**. No set can be a proper subset of itself, for we would require an element in the set that is not in the set! This is obviously impossible.

Since the empty set is a subset of every set, even itself, (d) is **true**.

Solution 1.3

Each of these sets is given by enumeration below.

$Y = \{\text{'a'}, \text{'d'}, \text{'e'}, \text{'r'}\}$.

$X = \{\text{'d'}, \text{'r'}, \text{'e'}, \text{'a'}\}$.

The required truth values are:

(a) **true** (each character in Y also appears in X);

(b) **true** ($X \subseteq Y$ is also **true** – the sets contain exactly the same characters);

(c) **false** ($Y \subseteq X$ is **true**, but so is $Y == X$).

Solution 1.4

$\{\}, \{4\}, \{6\}, \{1\}, \{4, 6\}, \{4, 1\}, \{6, 1\}$.

Solution 2.1

(a) $A \cup B = \{1, 2, 3, 4, 7, 5, 6\}$.

(b) $A \cap B = \{3\}$.

(c) $A - B = \{1, 2, 4, 7\}$.

Solution 2.2

The two sets can be enumerated as: $E = \{\text{'s'}, \text{'e'}, \text{'n'}, \text{'t'}, \text{'c'}\}$ and $F = \{\text{'l'}, \text{'e'}, \text{'n'}, \text{'g'}, \text{'t'}, \text{'h'}\}$.

Then: (a) (i) $E \cup F = \{\text{'s'}, \text{'e'}, \text{'n'}, \text{'t'}, \text{'c'}, \text{'l'}, \text{'g'}, \text{'h'}\}$; (ii) $E \cap F = \{\text{'e'}, \text{'n'}, \text{'t'}\}$;

(iii) $E - F = \{\text{'s'}, \text{'c'}\}$.

(b) The cardinalities are: for E , 5; for F , 6; for $E \cup F$, 8; for $E \cap F$, 3.

Solution 2.3

(a) (i) We have $B \cup C = \{0, 4, 5, 9, 6, 7\}$.

So $A - (B \cup C) = \{1, 2\}$.

(ii) We have $A - B = \{1, 2, 6\}$ and $A - C = \{1, 2, 5\}$.

So $(A - B) \cap (A - C) = \{1, 2\}$.

(b) See Figure S1.

(c) x will be in $A - (B \cup C)$ if (and only if) x is in the set A and x is not in the set $B \cup C$; that is, x is not in B and x is not in C . So the proposition

$x \in (A - (B \cup C))$ will have the same truth value as the proposition

$(x \in A) \wedge (\text{NOT}(x \in B)) \wedge (\text{NOT}(x \in C))$.

Notice how this corresponds to Figure S1, where the shaded area includes those items that are in A , not in B and not in C .

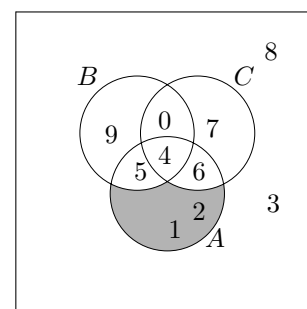


Figure S1 $A - (B \cup C)$

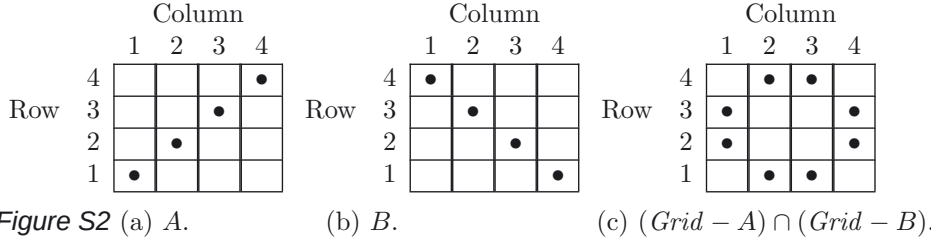
Solution 3.1

The power set of A is enumerated below. It has cardinality 4.

$\{\{\}, \{a\}, \{b\}, \{a, b\}\}$

Solution 3.2

See Figure S2.



Solution 3.3

The states of the variables are: for A , $\{a, c, t\}$; for B , $\{a, d, s\}$. Then

- (a) $A.card()$ is 3.
- (b) $B.union(A)$ is $\{a, d, s, c, t\}$.
- (c) $A.intersect(B)$ is $\{a\}$.
- (d) $A.minus(B)$ is $\{c, t\}$.
- (e) $B.minus(A)$ is $\{d, s\}$.
- (f) $A.cartProd(B)$ is $\{(a, a), (a, d), (a, s), (c, a), (c, d), (c, s), (t, a), (t, d), (t, s)\}$.

Solution 3.4

(a) See Figure S3.

(a) (i) **true**. (ii) This is undefined, since *contains* requires as its argument a value that is a potential member of the receiver set. So for *contains* the argument here should be of type **Pair of Int, Int**, not of type **Set**. (iii) **false**. (For example, AxC contains $(3, 6)$, which is not in $aGrid$.)

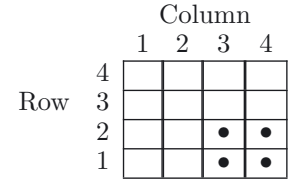


Figure S3

Solution 4.1

(a) (i) The predefined set is $SmInt$ and the selector predicate is $x < 2$.

The set is $\{0, 1\}$.

(ii) The predefined set is $Char$ and the selector predicate is $c > 'z'$.

The set is $\{\{, |, \}, \sim\}$.

(b) (i) $\{x \text{ in } SmInt : (x < 3)\} \cup \{x \text{ in } SmInt : (x > 8)\}$.

The set is $\{0, 1, 2, 9\}$.

(ii) $\{c \text{ in } Char : (c > 'R')\} \cap \{c \text{ in } Char : (c < 'W')\}$. The set is $\{S, T, U, V\}$.

Solution 4.2

There are various ways of describing this set. One is

$\{(x, y) \text{ in } Grid : (x < 3) \wedge (y < 3)\}$.

Solution 4.3

We can use an approach similar to that used in the comment on Activity 4.3. However, there are a number of differences. We need a variable that will hold a set of *pairs* of integers. We need to iterate through all possible values of both x and y (two variables), so we will need a **for** loop within a **for** loop. We only need to iterate each of x and y from 1 to 4. We need a variable to hold a pair of integers before it is added to the set *mySet*.

A suitable code fragment is given below.

```
var mySet in Set of Pair of Int,Int
var p in Pair of Int,Int
var x in Int
var y in Int
for (x <-- 1 to 4)
{
  for (y <-- 1 to 4)
  {
    if (x - y == 1) then
    {
      p.setPair(x,y)
      mySet.addElement(p)
    }
  }
}
```

Index

- cardinality, 8
- Cartesian product, 25
 - component of, 25
- complement, 22
- De Morgan's Laws, 28
- disjoint sets, 20
- empty set, 8
- Grid*, 27
- ordered pair, 25
- power set, 24
- predefined set, 38
- predicate, 36
 - selector, 38
- proposition, 9
 - truth value of a , 9
- relative complement, 22
- Russell's paradox, 22
- set, 6
 - element of a , 9
 - empty, 8
 - given by enumeration, 6
 - member, 9
- set comprehension, 36
- set difference, 21
- set equality, 14
- set intersection, 20
- Set of X** , 31
- set union, 17
- subset, 11
 - proper, 15
- typing, 38
- Venn diagram, 19